

DATA 571



Review Session

DATA 571: Resampling and Regularization

AUTHOR

Dr. Irene Vrbik

AFFILIATION

Masters of Data Science UBCO

Plan for Today [🔗](#)

- Big-picture check-in
- Concept review:
 - Resampling (validation, CV, bootstrap)
 - Trees, bagging, random forests, boosting
 - Ridge & LASSO
- Builds on **DATA 570**:
 - You already know linear/logistic regression, KNN, basic trees, etc.
- 571 is about:
 - Getting **reliable estimates of test error**
 - Controlling **bias–variance trade-off**
 - Making **robust models**: resampling, ensembles, regularization

Goal: you can explain *when and why* you'd use each method, **not** just run the code.

General Themes

Throughout 570 and 571 we discussed several themes that are worth reviewing.

Data Splitting

In DATA 571, almost everything we do comes back to one central question:

How well will my model perform on new, unseen data?

This sounds simple, but it's surprisingly easy to answer *incorrectly*.

When we fit a model using a dataset, the model is allowed to “see” those observations. Even if the relationship we learn is completely meaningless, the model can still appear to perform well on the data it was trained on. This is not because the model is good – it's because the model has already been exposed to those observations.

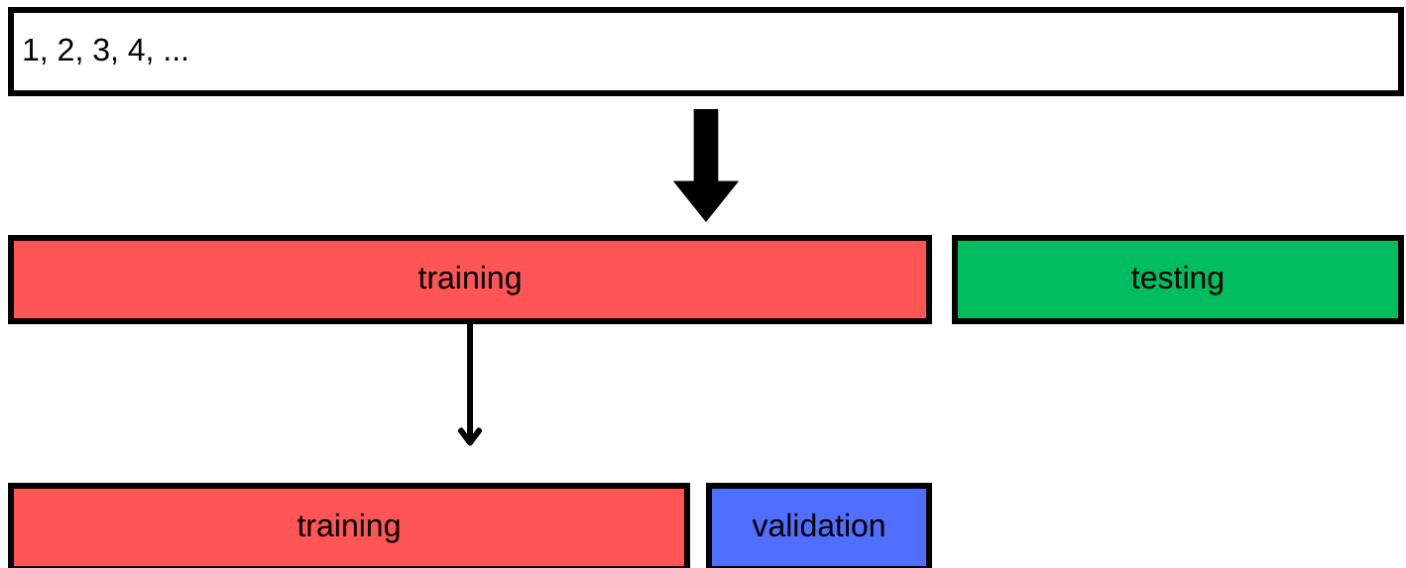
This is why **training error is optimistic**: it systematically underestimates how much error we'll see in the real world.

Data splitting is our first and most important tool for combating this optimism.

Training vs Test Data

The simplest way to estimate how a model will generalize is to set aside part of the data and pretend we never saw it during model fitting. In other words, we allocate data to certain tasks

- The **training set** is used to *fit* the model.
- The **test set** is used only once, at the very end, to *evaluate performance*.



A schematic display of the training-test set approach. A set of n observations are randomly split into a training set (shown in red) and a test set (shown in green). The statistical learning method is fit on the training set, and its performance is evaluated on the test set.

Because the **test set** was *not* used during model fitting, error measured on the test set gives us an estimate of test error – the error we would expect when the model is deployed on future data. The test set is sacred. If you use it to make modeling decisions, it is no longer a test set.

Important

The **test set** should not be used in the training or tuning process.

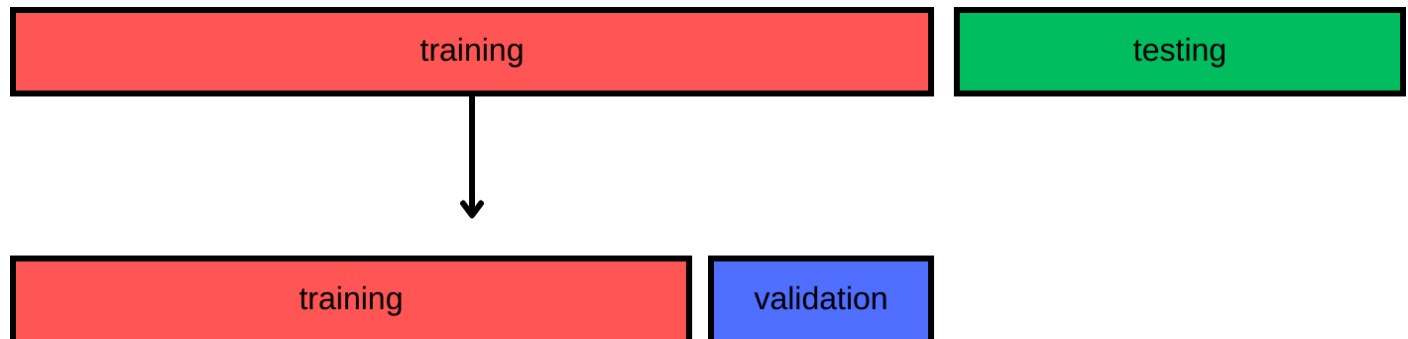
Question: How much data should be allocated to the training and test sets?

There is no single “correct” train–test split. The right allocation depends on how much data you have and what you need the data for. What matters is not the exact percentages, but that each split is large enough to do its job well. In practice, a common split is 80% training / 20% test.

Validation Approach

If we don't want to use the test set to tune our model, we may be tempted to simply split the training data again to produce:

- a **training set** used to *fit* the model
- a **validation set** to *tune* hyperparameters
- a **test set** to *evaluate* the final performance.



A schematic display of the validation set approach. A set of n observations are randomly split into a training set (shown in red) and a validation set (shown in blue). The statistical learning method is fit on the training set, and its performance is evaluated on the validation set.

A single training-validation split allows us to tune model parameters without using the test set, but it introduces an important source of variability. The validation error depends on which observations happen to be included in the validation set, and different random splits can lead to different tuning decisions.

To illustrate this, we repeat the validation-set procedure ten times using different random splits of the Auto data. For each split, we compute the validation mean squared error for KNN regression across a range of values of k , and select the value of k that minimizes validation error.

Example:KNN with a single validation set

```
library(ISLR2) # for Auto data
library(FNN)   # for KNN algorithm

# load and prep data
data(Auto)
(Auto <- na.omit(Auto))
```

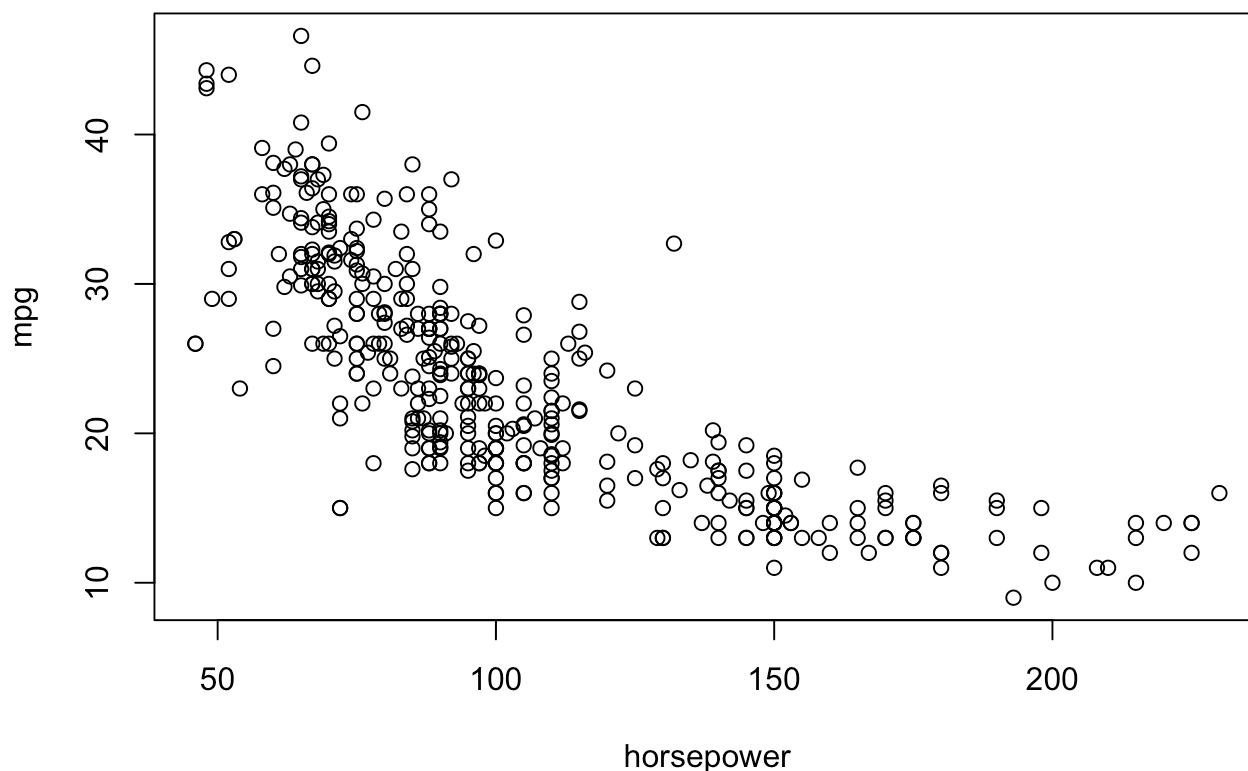
	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin
	<dbl>	<int>	<dbl>	<int>	<int>	<dbl>	<int>	<int>
1	18.0	8	307.0	130	3504	12.0	70	1
2	15.0	8	350.0	165	3693	11.5	70	1
3	18.0	8	318.0	150	3436	11.0	70	1
4	16.0	8	304.0	150	3433	12.0	70	1
5	17.0	8	302.0	140	3449	10.5	70	1
6	15.0	8	429.0	198	4341	10.0	70	1
7	14.0	8	454.0	220	4354	9.0	70	1
8	14.0	8	440.0	215	4312	8.5	70	1

	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin
	<dbl>	<int>	<dbl>	<int>	<int>	<dbl>	<int>	<int>
9	14.0	8	455.0	225	4425	10.0	70	1
10	15.0	8	390.0	190	3850	8.5	70	1

1-10 of 392 rows | 1-9 of 10 columns

Previous **1** [2](#) [3](#) [4](#) [5](#) [6](#) ... [40](#) [Next](#)

```
plot(mpg ~ horsepower, data = Auto)
```



Let's split into a random train/validate/test set

```
set.seed(1)
n <- nrow(Auto)

train_frac = 0.5 # 50% training
val_frac = 0.3 # 30% validation
test_frac = 0.2 # 20% testing

test_idx <- sample(1:n, size = floor(test_frac * n))
train_idx <- setdiff(1:n, test_idx)
val_idx <- sample(train_idx, size = floor(val_frac*n))
train_idx <- setdiff(train_idx, val_idx)
```

► Code

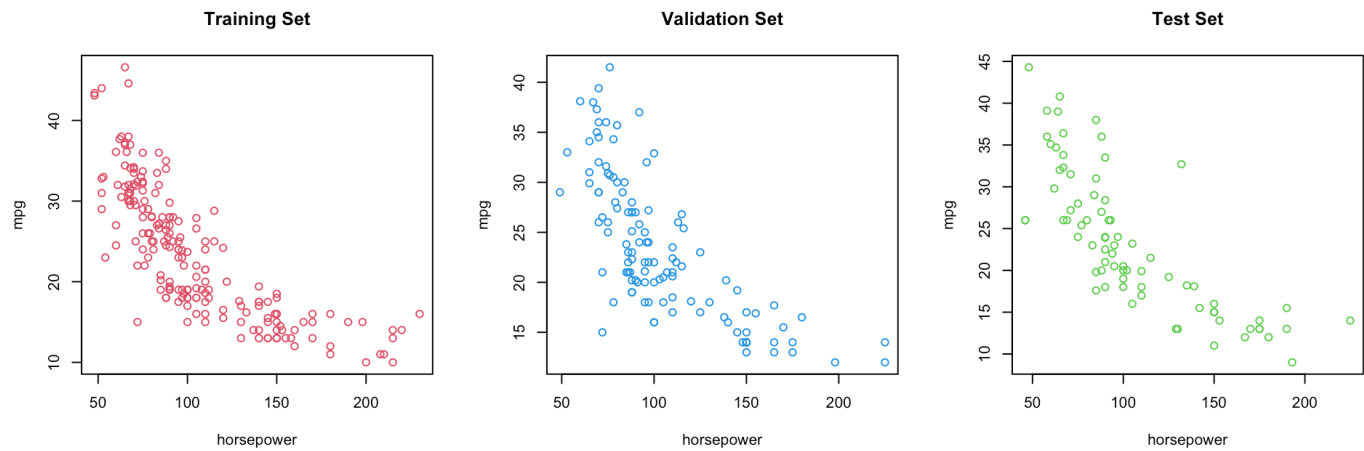


Figure 1

► Code

KNN Regression on Auto Data mpg ~ horsepower

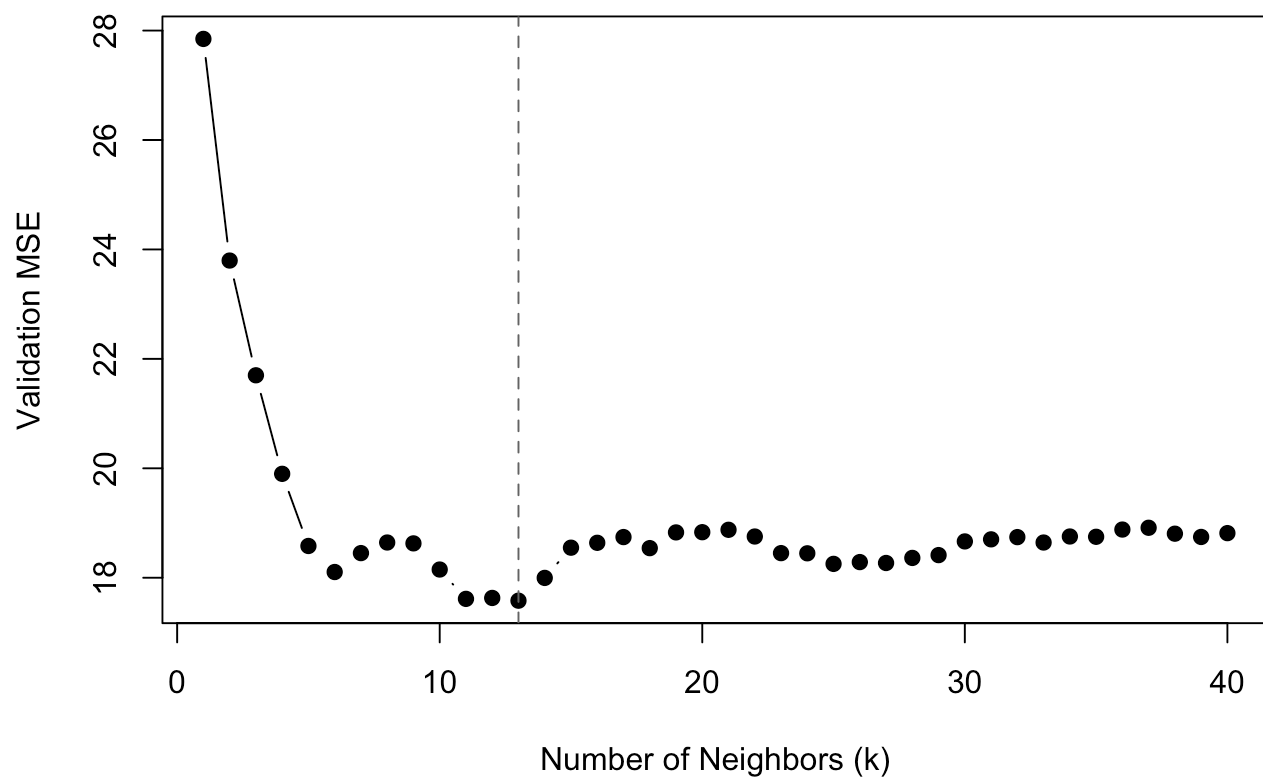


Figure 2: Validation error curve for KNN regression on the Auto data. The dashed vertical line indicates the chosen k for this particular split.

The optimal value of k depends on the validation split. For this particular split, the optimal value was `best_k`. If we to use a different seed however, the results could be different.

► Code

KNN Regression on Auto Data mpg ~ horsepower

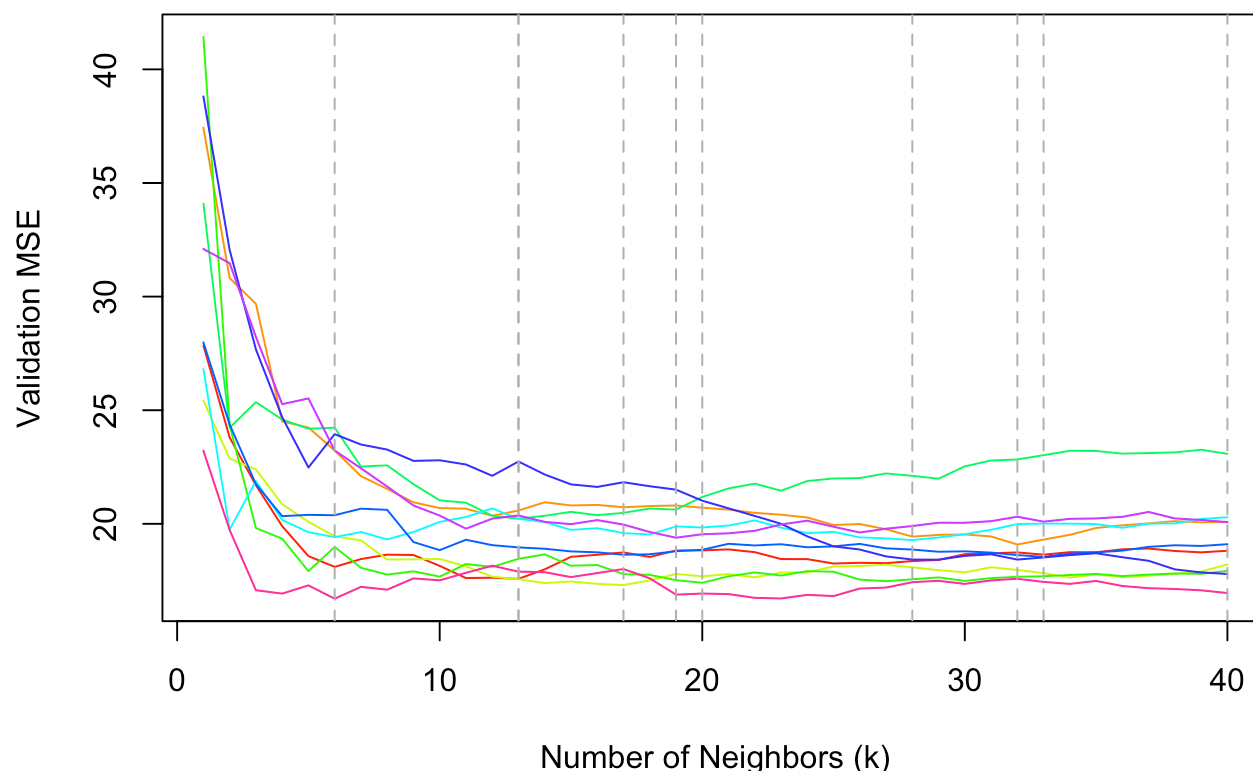


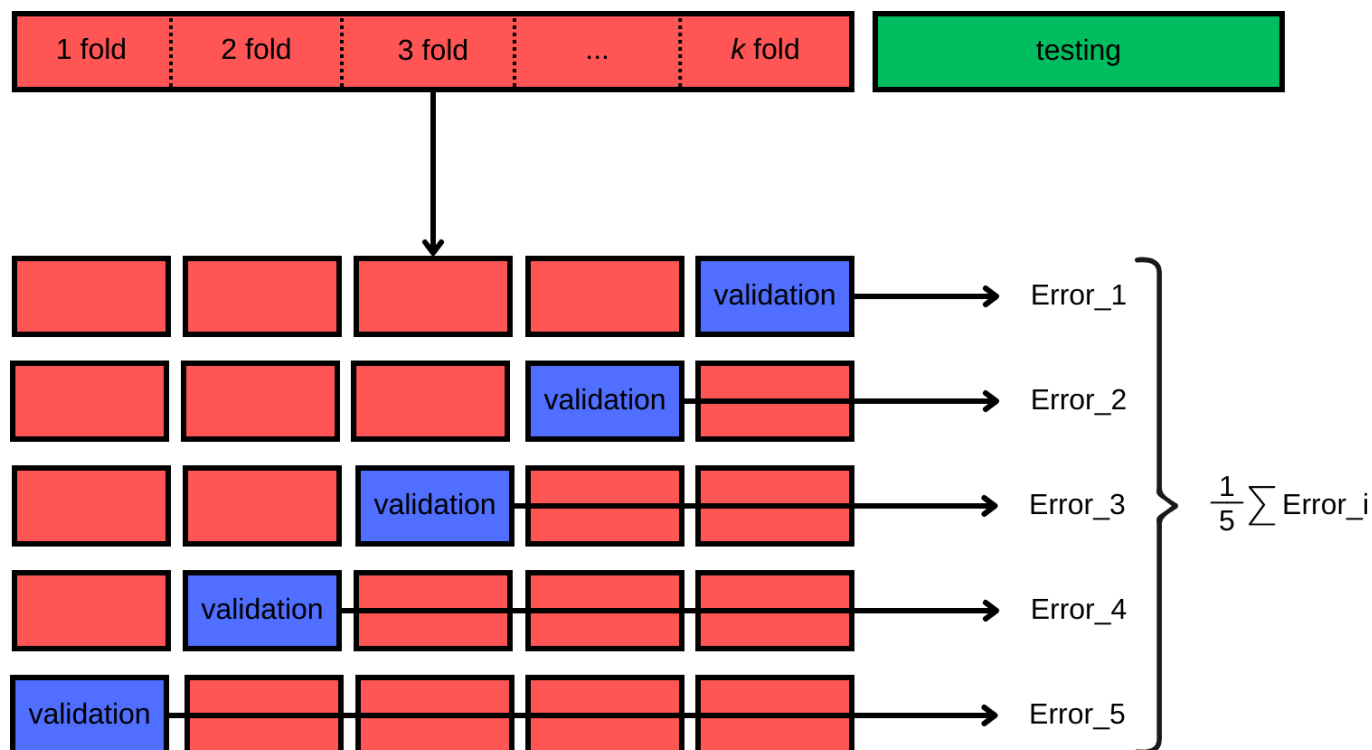
Figure 3: Validation error curves for KNN regression on the Auto data, obtained using ten different random training-validation splits. The variability across curves illustrates the instability of the single validation-set approach.

The resulting curves reveal substantial variability across splits, and the selected “optimal” value of k differs from run to run¹. This instability is not due to changes in the model or data, but rather to randomness in the validation set itself.

This motivates the use of cross-validation, which averages validation performance across many splits and produces more stable and reliable tuning decisions.

Cross-Validation

Cross-validation (CV) is an integral statistical method for tuning our models. It can be used to estimate the test error associated with a given statistical learning method in order to evaluate its performance, or to select the appropriate level of flexibility. It involves training the model on a portion of the data and validating it on another, then repeating this process multiple times with different splits to get a more reliable estimate of how well the model will generalize to new, unseen data.



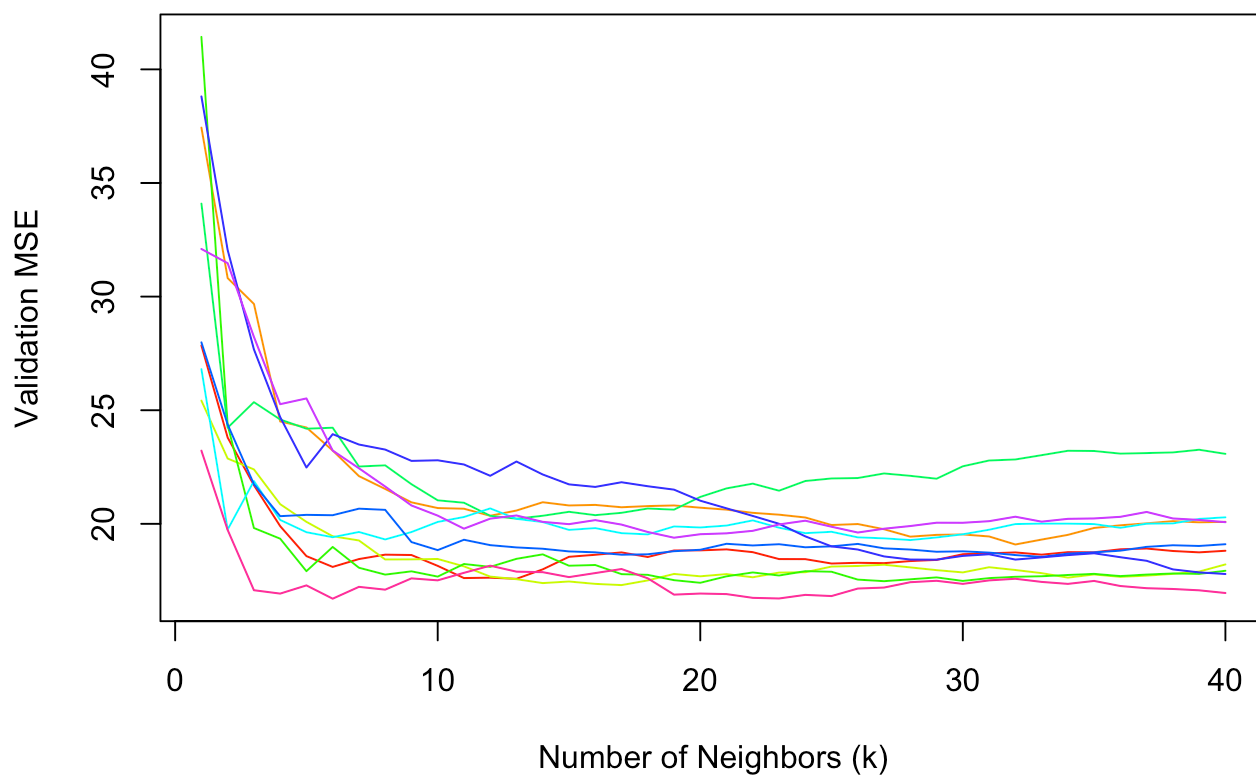
A schematic display of 5-fold CV. A set of n observations is randomly split into five non-overlapping groups (aka “folds”). The model is trained five times: in each fit, one fold (shown in blue) is held out for testing and the model is trained on the remaining folds (shown in red). The CV error is estimated by averaging the five performance scores

Example: Repeated using CV

Instead of selecting a single validation set, we now use 10-fold cross-validation on the training data to select the optimal value of k for the KNN algorithm applied to the **Auto** data. To highlight stability, we repeat the entire cross-validation procedure ten times, each time using a different random fold assignment.

► Code

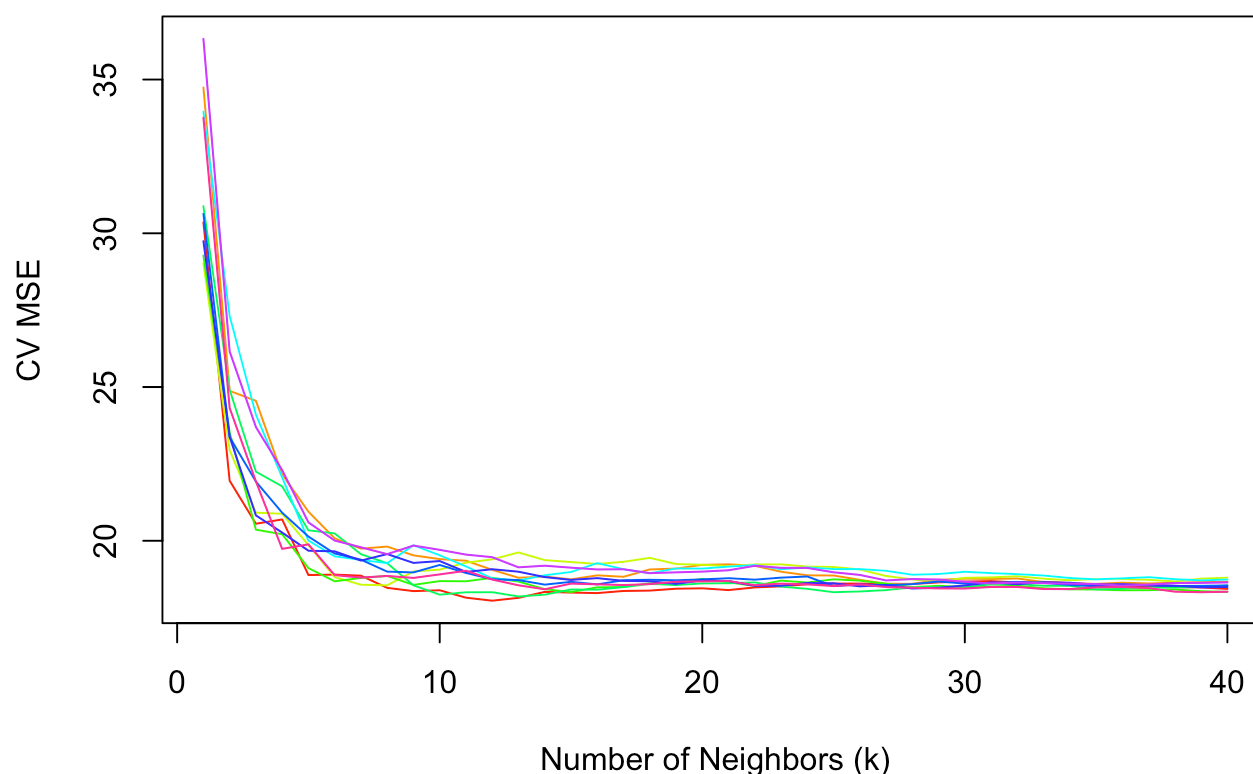
KNN Regression on Auto Data mpg ~ horsepower



(a) Validation error curves for KNN regression on the Auto data, obtained using ten different random training-validation splits. The variability across curves illustrates the instability of the single validation-set approach.

KNN Regression with 10-Fold CV

mpg ~ horsepower



(b) Cross-validation error curves for KNN regression on the Auto data, obtained from ten independent runs of 10-fold cross-validation. Vertical dashed lines indicate the optimal k selected in each run.

Figure 4

Cross-validation improves on the validation-set approach by averaging model performance over many different training-validation splits. This substantially reduces the variability of error estimates and leads to more stable tuning decisions. As demonstrated in this example, the cv error estimates exhibit substantially greater stability when compared to the validation-set approach.

Choosing the number of folds

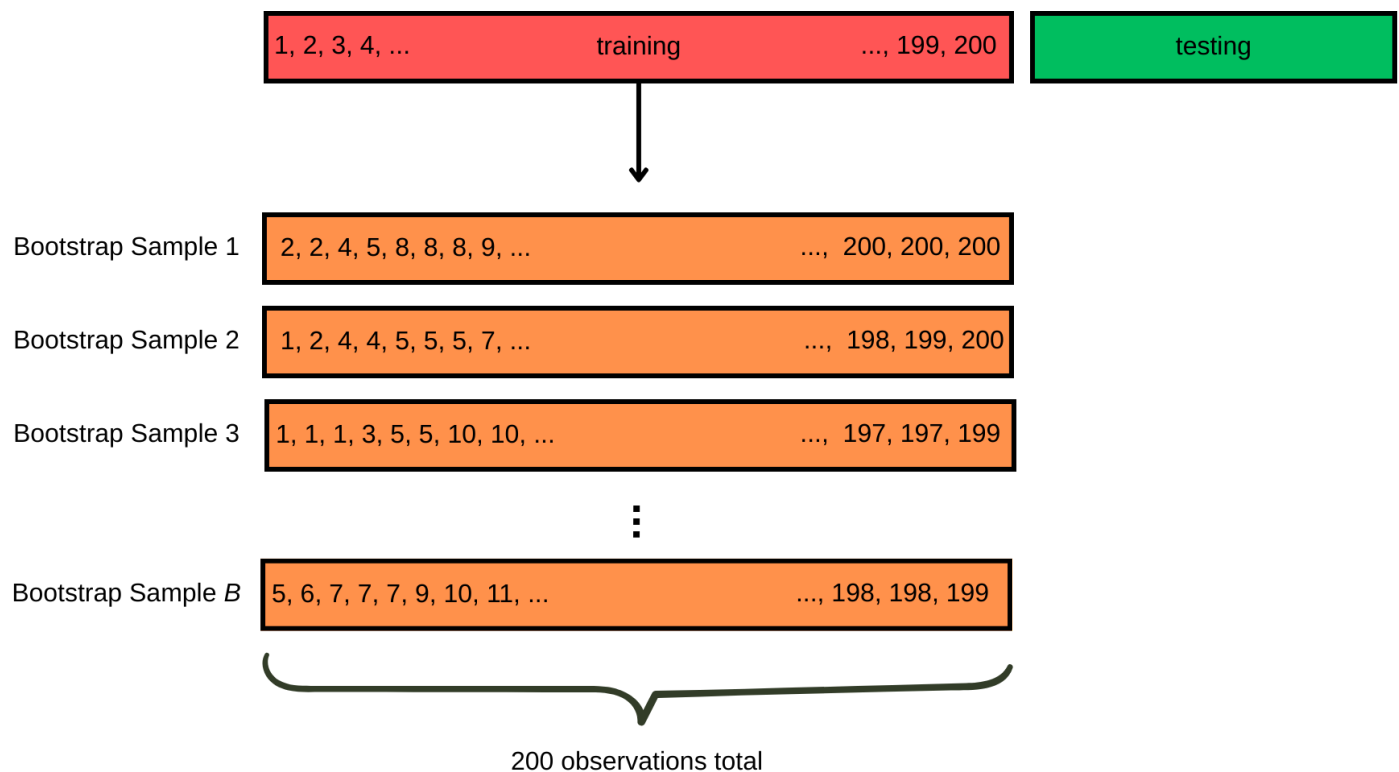
A special case is leave-one-out cross-validation (LOOCV) when k (the number of folds) is equal to the number of samples. While there is no formal rule, typical choices for k are 5 or 10.

Bootstrap

While cross-validation is the default tool for selecting tuning parameters such as the number of neighbors in KNN, the depth of a tree, or the penalty parameter in ridge and LASSO regression, it is not the only resampling method available. A closely related but conceptually different approach is the bootstrap.

Rather than splitting the data into distinct training and validation sets, the bootstrap repeatedly resamples the dataset with replacement. This resampling scheme produces training sets that are highly similar to the original

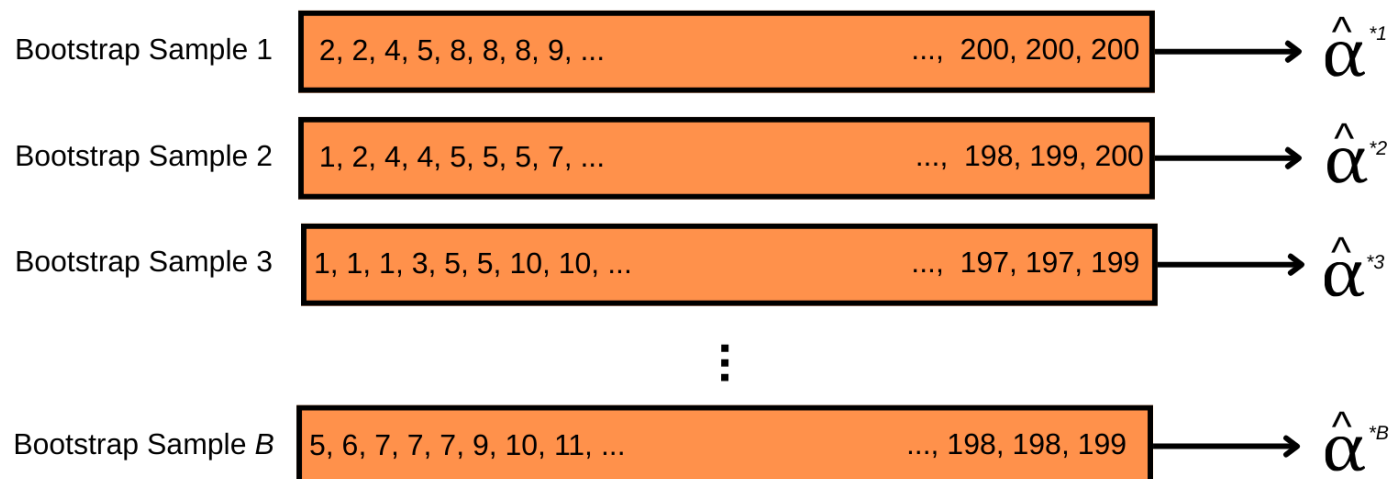
dataset but introduces additional randomness through duplication and omission. Each bootstrap sample has the same size as the original dataset, but due to sampling with replacement, some observations appear multiple times while others are left out entirely.



Schematic illustration of the bootstrap procedure. The original training set is repeatedly resampled with replacement to form bootstrap samples of the same size as the original data.

Bootstrap for Bias and Variance Estimation

One of the main strengths of the bootstrap is that it allows us to approximate the sampling distribution of an estimator. From this approximate sampling distribution, we can estimate both bias and variance.



Bootstrap resampling for bias and variance estimation. Sampling with replacement from the training data produces multiple bootstrap estimates of the estimator, whose empirical distribution approximates the true sampling distribution. This distribution is used to estimate both bias and variance.

The B bootstrap samples produce B estimates of α , denoted:

$$\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \dots, \hat{\alpha}^{*B}$$

Given these estimates, we can compute an estimate of the standard error of $\hat{\alpha}$ using the following formula:

$$\widehat{SE}_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B \left(\hat{\alpha}^{*b} - \frac{1}{B} \sum_{b=1}^B \hat{\alpha}^{*b} \right)^2}$$

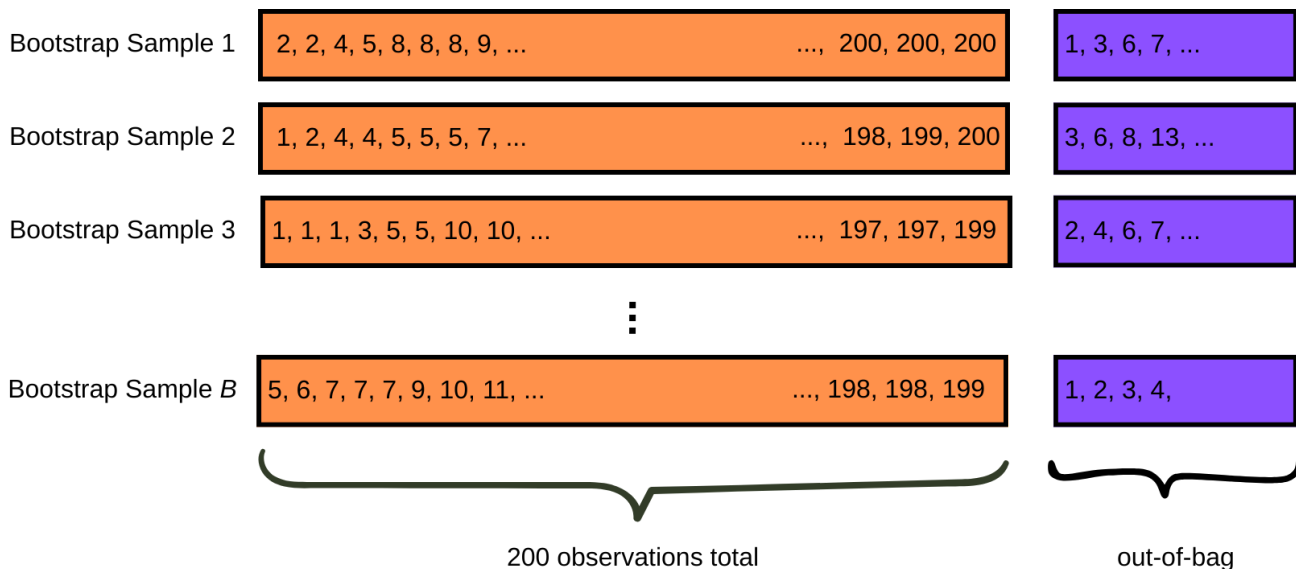
and the bias of $\hat{\alpha}$ using the following formula:

$$\widehat{Bias}_B(\hat{\alpha}) = \bar{\alpha}^* - \hat{\alpha}.$$

where $\hat{\alpha}$ is the estimate of the parameter α computed on the original (non-resampled) training set.

Bootstrap Estimate of Error

On average, about 63.2% of the original observations appear in a given bootstrap sample. The remaining 36.8%, known as out-of-bag (OOB) observations, can be used to estimate prediction error.



Each bootstrap sample contains duplicated observations and omits others, with omitted observations serving as out-of-bag data for error estimation. The testing set remains untouched throughout this process

Here's how we saw *bootstrap* in use:

- **Estimation of bias, standard error, and confidence intervals:** bootstrap resampling can be used to estimate bias, standard errors and confidence intervals around model parameters. By repeatedly resampling the dataset and fitting the model, you can assess the variability of the estimated parameters. Know how to read/write a program for doing this.
- **Bagging (Bootstrap Aggregating):** the bootstrap can be employed in ensemble methods like bagging decisions trees. More generally bagging involves training multiple instances of the same learning algorithm on different bootstrap samples of the training data and combining their predictions to reduce variance and improve generalization.

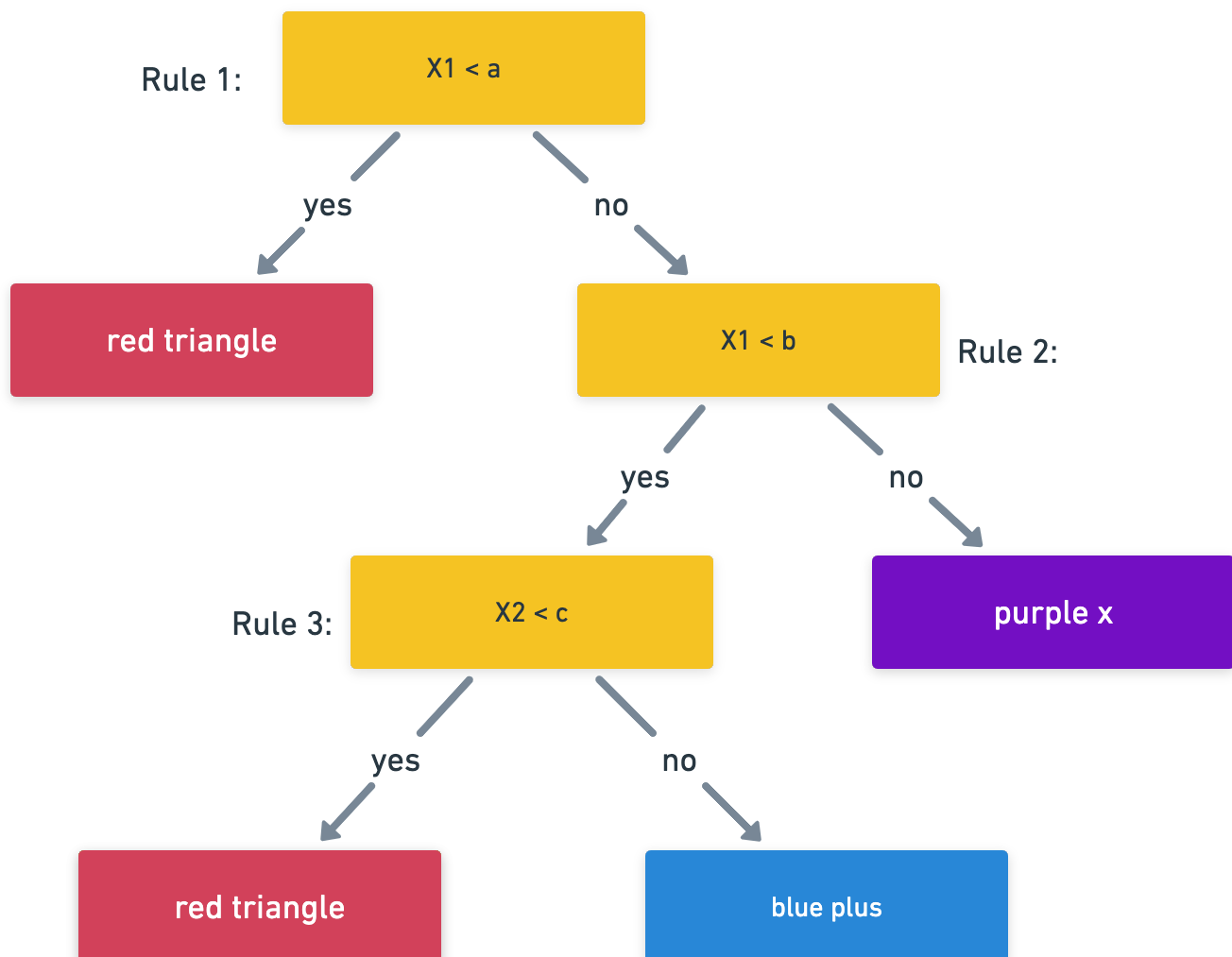
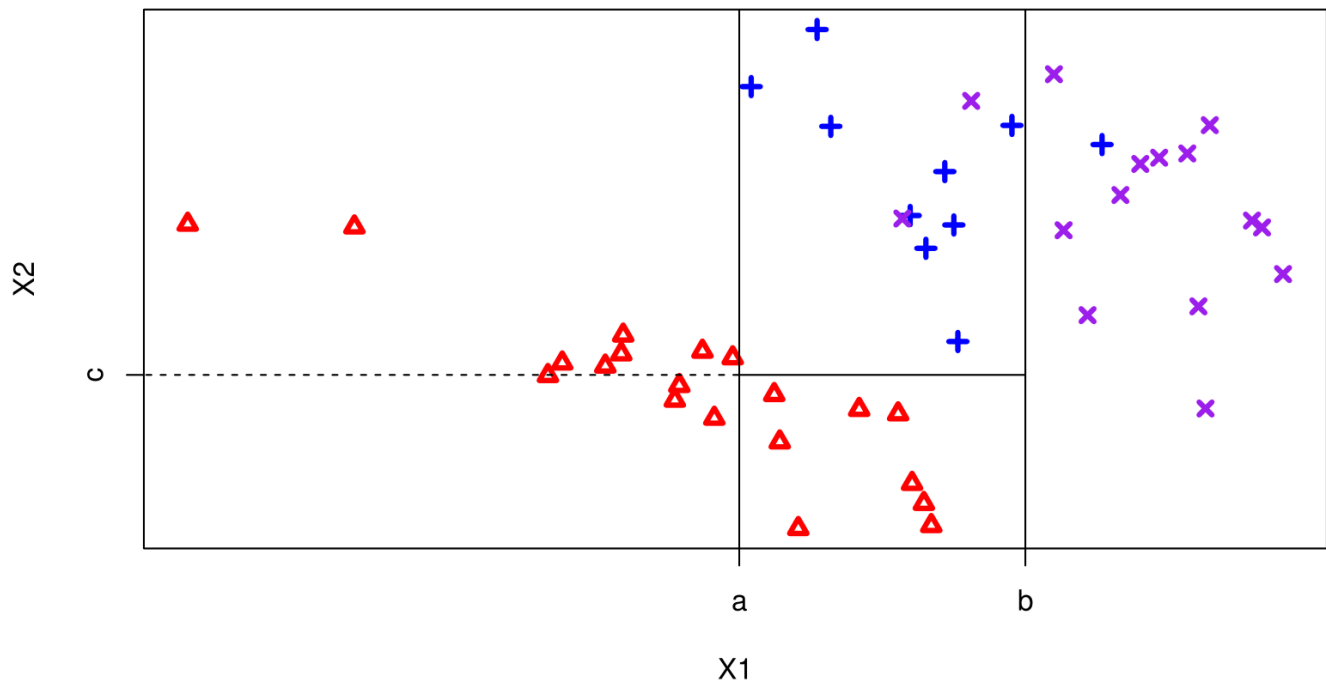
- **Random Forests:** (an extension of Bagging, where each tree is constructed using a random subset of features at each split, introducing an additional layer of randomness) used bootstrap sampling to create multiple decision trees. Each tree is trained on a bootstrap sample, and their predictions are combined through voting or averaging.
- **Feature Importance:** In bagging and random forests the bootstrap inherently provides an estimate the importance of features in a model. The importance is inferred by assessing how the model performance changes with and without each feature.
- **Out-of-bag error estimates:** In ensemble methods like Random Forests and Bagging, out-of-bag (OOB) estimates of error provide a way to assess the model's performance without the need for a separate validation set or cross-validation. Here's how it works:
 - During the training of each tree in the ensemble, a bootstrap sample (a random sample with replacement) is drawn from the original dataset; the instances from the training set that are *not* sampled are called the “out-of-bag” samples.
 - For each observation in the training set we predict its target variable using only the trees for which it was an out-of-bag sample and calculate the error (e.g., classification error or mean squared error).

Classification and Regression Trees

Classification and Regression Trees (AKA *CART* or *decision trees*) involve *partitioning* (AKA *stratifying* or *segmenting*) the predictor/feature space by simple binary splitting rules.

The model builds a tree structure to make predictions based on recursive binary splits of the input features. Each internal node represents a decision, and each leaf node represents the prediction (e.g. value or class label).

Since the set of splitting rules used to segment the predictor space can be summarized in a tree, these types of approaches are known as decision tree methods.



The complexity of a tree is determined by its size, typically measured by the number of terminal nodes (leaves).

Objective (for classification)

The goal of a decision tree for classification is to partition the input space into distinct regions, each corresponding to a specific class or category. **Maximizes** information gain or **Gini** impurity during tree construction. We make a prediction for an observation using the mode response value (i.e. majority vote) for that “region” to which it belongs.

Objective (for regression)

Minimize the RSS (via recursive binary splitting) at each node. We make a prediction for an observation using the mean response value for that “region” to which it belongs.

Growing a tree

1. First we grow a tree using recursive binary splitting until some stopping criterion is met.
 - regression uses RSS for this
 - classification grows using the Gini index
2. Prune the tree: since the tree in the last time is probably too bushy and has high variance we trim it back using cross-complexity pruning. (use cross-validation to determine by how much)

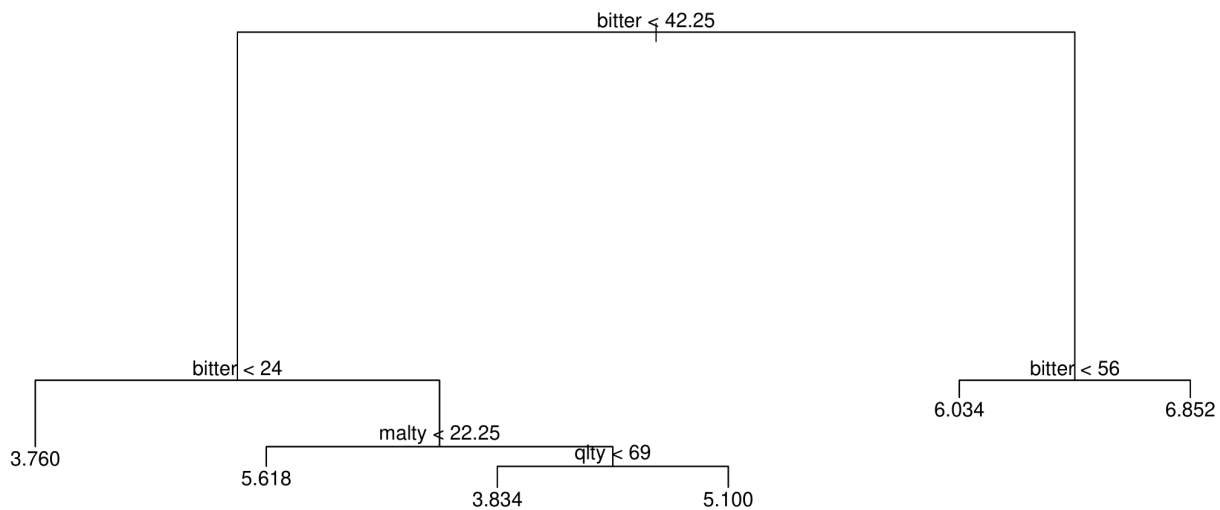
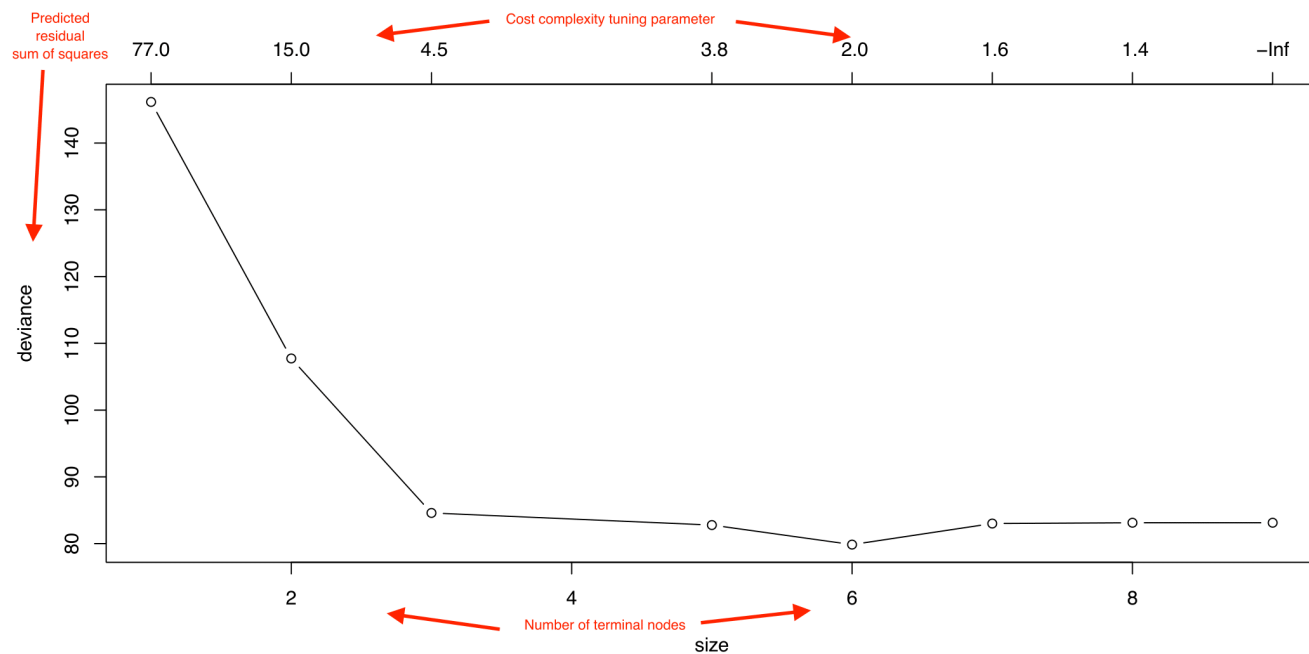
We consider a sequence of trees indexed by a non-negative tuning parameter α . For every value of α there corresponds a subtree, $T \in T_0$, such that the following is minimized:

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

- Here $|T|$ is the number of terminal nodes in the tree,
- R_m is the region corresponding to the m th terminal node
- $\hat{y}_{R_m}$ is the mean of the training observations in R_m .

Tuning Parameter

- When $\alpha = 0$, then the subtree T will simply equal T_0
- However, as α increases, there is a price to pay for having a tree with many terminal nodes, and so the penalized RSS will tend to be minimized for a smaller subtree.
- In that way, α controls the trade-off between the subtree's complexity and its fit to the training data.
- The tuning parameter α is selected using cross-validation.



Trees have one aspect that prevents them from being ideal tool for predictive learning, namely **inaccuracy**.

(ESL)

- R functions: `tree`, `cv.tree`, `prune.tree` from the **tree** package
- Know how to read the trees and make predictions from them

Pros

- highly interpretable models
- can handle many types of predictors (continuous, categorical, incomplete (with **NA**s), ...)
- Suitable for problems with complex, non-linear relationships.

Cons

- high variance (small changes in the data can drastically change the structure of the tree or rules)
 - Prone to overfitting.
- poor prediction performance

That said, they are still useful since:

1. they introduce the important concept of shrinkage
2. are used as the base learners in bagging, random forests, and boosting

Example: Carseats

We first use classification trees to analyze the **Carseats** data set. In these data, **Sales** is a continuous variable, and so we begin by recoding it as a binary variable. We use the `ifelse()` function to create a variable, called **High**, which takes on a value of **Yes** if the **Sales** variable exceeds 8, and takes on a value of **No** otherwise.

```
library(tree)
library(ISLR2)
attach(Carseats)
High <- factor(ifelse(Sales <= 8, "No", "Yes"))
Carseats <- data.frame(Carseats, High)
Carseats
```

	Sales <dbl>	CompPrice <dbl>	Income <dbl>	Advertising <dbl>	Population <dbl>	Price <dbl>	ShelveLoc <fct>	A... <dbl>
1	9.50	138	73	11	276	120	Bad	42
2	11.22	111	48	16	260	83	Good	65
3	10.06	113	35	10	269	80	Medium	59
4	7.40	117	100	4	466	97	Medium	55
5	4.15	141	64	3	340	128	Bad	38
6	10.81	124	113	13	501	72	Bad	78
7	6.63	115	105	0	45	108	Medium	71
8	11.85	136	81	15	425	120	Good	67
9	6.54	132	110	0	108	124	Medium	76

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc	A...
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<fct>	<dbl>
10	4.69	132	113	0	131	124	Medium	76

1-10 of 400 rows | 1-9 of 13 columns

Previous **1** [2](#) [3](#) [4](#) [5](#) [6](#) ... [40](#) [Next](#)

We start by splitting the observations into a training set and a test set:

```
set.seed(2)

n = nrow(Carseats)

# training
train_prop = 0.5
train <- sample(1:nrow(Carseats), floor(train_prop*n))
Carseats.test <- Carseats[-train, ]
High.test <- High[-train]
```

We now use the `tree()` function to fit a classification tree in order to predict `High` using all variables but `Sales`. The syntax of the `tree()` function is quite similar to that of the `lm()` function

```
tree.carseats <- tree(High ~ . - Sales, Carseats,
  subset = train)
summary(tree.carseats)
```

Classification tree:

```
tree(formula = High ~ . - Sales, data = Carseats, subset = train)
```

Variables actually used in tree construction:

```
[1] "Price"      "Population" "ShelveLoc"  "Age"        "Education"
[6] "CompPrice" "Advertising" "Income"     "US"
```

Number of terminal nodes: 21

Residual mean deviance: 0.5543 = 99.22 / 179

Misclassification error rate: 0.115 = 23 / 200

The `summary()` function lists the variables that are used as internal nodes in the tree, the number of terminal nodes, and the (training) error rate. The deviance reported in the output of `summary()` is given by

$$-2 \sum \sum n_{mk} \log \hat{p}_{mk}$$

where n_{mk} is the number of observations in the m th terminal node that belong to the k th class. Hence, a small deviance indicates a tree that provides a good fit to the (training) data.

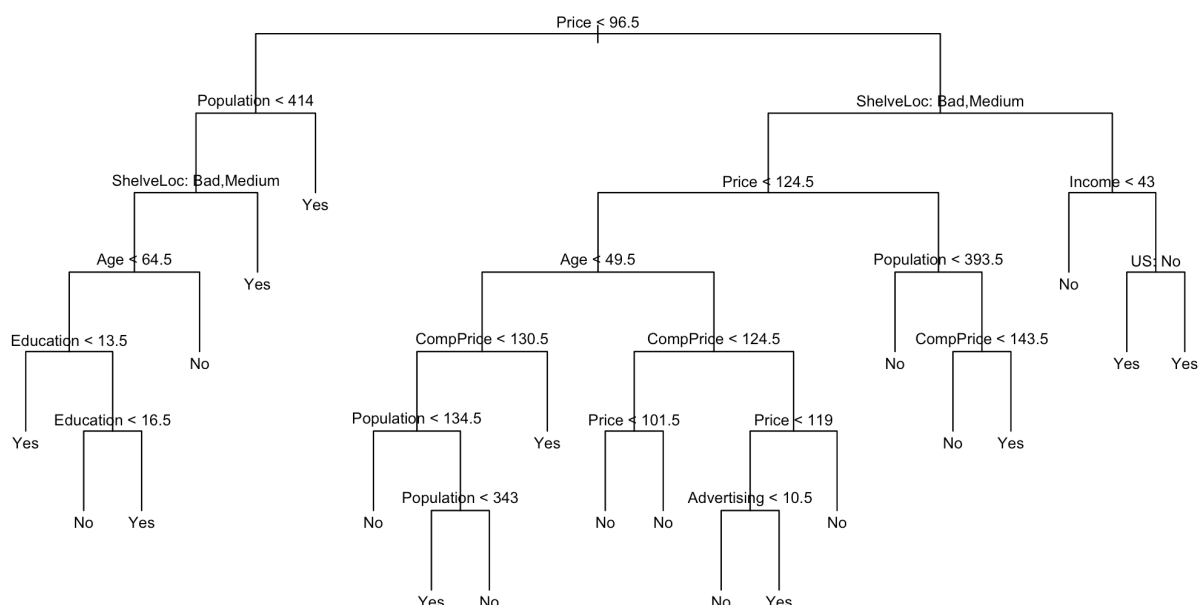
One of the most attractive properties of trees is that they can be graphically displayed. We use the `plot()` function to display the tree structure, and the `text()` function to display the node labels. The argument `pretty = 0` instructs R to include the category names for any qualitative predictors, rather than simply displaying a letter for each category.

```

plot(
  tree.carseats,
  type = "uniform"
)

text(
  tree.carseats,
  pretty = 0,
  cex = 0.8          # smaller text
)

```



The most important indicator of Sales appears the Price.

If we just type the name of the tree object, R prints output corresponding to each branch of the tree. Branches that lead to terminal nodes are indicated using asterisks.

```
tree.carseats
```

```
node), split, n, deviance, yval, (yprob)
* denotes terminal node
```

```

1) root 200 270.000 No ( 0.59500 0.40500 )
 2) Price < 96.5 40 47.050 Yes ( 0.27500 0.72500 )
   4) Population < 414 31 40.320 Yes ( 0.35484 0.64516 )
    8) ShelveLoc: Bad,Medium 25 34.300 Yes ( 0.44000 0.56000 )
     16) Age < 64.5 17 20.600 Yes ( 0.29412 0.70588 )
      32) Education < 13.5 7 0.000 Yes ( 0.00000 1.00000 ) *
```

```

33) Education > 13.5 10 13.860 Yes ( 0.50000 0.50000 )
66) Education < 16.5 5 5.004 No ( 0.80000 0.20000 ) *
67) Education > 16.5 5 5.004 Yes ( 0.20000 0.80000 ) *
17) Age > 64.5 8 8.997 No ( 0.75000 0.25000 ) *
9) ShelveLoc: Good 6 0.000 Yes ( 0.00000 1.00000 ) *
5) Population > 414 9 0.000 Yes ( 0.00000 1.00000 ) *
3) Price > 96.5 160 201.800 No ( 0.67500 0.32500 )
6) ShelveLoc: Bad,Medium 135 154.500 No ( 0.74074 0.25926 )
12) Price < 124.5 82 107.700 No ( 0.63415 0.36585 )
24) Age < 49.5 34 45.230 Yes ( 0.38235 0.61765 )
48) CompPrice < 130.5 21 28.680 No ( 0.57143 0.42857 )
96) Population < 134.5 6 0.000 No ( 1.00000 0.00000 ) *
97) Population > 134.5 15 20.190 Yes ( 0.40000 0.60000 )
194) Population < 343 7 5.742 Yes ( 0.14286 0.85714 ) *
195) Population > 343 8 10.590 No ( 0.62500 0.37500 ) *
49) CompPrice > 130.5 13 7.051 Yes ( 0.07692 0.92308 ) *
25) Age > 49.5 48 46.330 No ( 0.81250 0.18750 )
50) CompPrice < 124.5 28 14.410 No ( 0.92857 0.07143 )
100) Price < 101.5 8 8.997 No ( 0.75000 0.25000 ) *
101) Price > 101.5 20 0.000 No ( 1.00000 0.00000 ) *
51) CompPrice > 124.5 20 25.900 No ( 0.65000 0.35000 )
102) Price < 119 14 19.410 No ( 0.50000 0.50000 )
204) Advertising < 10.5 9 11.460 No ( 0.66667 0.33333 ) *
205) Advertising > 10.5 5 5.004 Yes ( 0.20000 0.80000 ) *
103) Price > 119 6 0.000 No ( 1.00000 0.00000 ) *
13) Price > 124.5 53 33.120 No ( 0.90566 0.09434 )
26) Population < 393.5 34 0.000 No ( 1.00000 0.00000 ) *
27) Population > 393.5 19 21.900 No ( 0.73684 0.26316 )
54) CompPrice < 143.5 13 7.051 No ( 0.92308 0.07692 ) *
55) CompPrice > 143.5 6 7.638 Yes ( 0.33333 0.66667 ) *
7) ShelveLoc: Good 25 31.340 Yes ( 0.32000 0.68000 )
14) Income < 43 7 8.376 No ( 0.71429 0.28571 ) *
15) Income > 43 18 16.220 Yes ( 0.16667 0.83333 )
30) US: No 6 8.318 Yes ( 0.50000 0.50000 ) *
31) US: Yes 12 0.000 Yes ( 0.00000 1.00000 ) *

```

Pruning

Understanding that these models are liable to overfit, we consider whether *pruning* the tree might lead to improved results. To find the optimal sized tree, we penalize the error rate according to the size of the tree.

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (1)$$

The function `cv.tree()` performs cross-validation in order to determine the optimal level of tree complexity. More specifically, **cost complexity pruning** is used in order to select a sequence of trees for consideration. We use the argument `FUN = prune.misclass` in order to indicate that we want the classification error rate to guide the cross-validation and pruning process, rather than the default for the `cv.tree()` function, which is deviance. The `cv.tree()` function reports the number of terminal nodes of each tree considered (size) as well as the

corresponding error rate and the value of the cost-complexity parameter used k , which corresponds to $\log \alpha$ in [Equation 1](#).

```
set.seed(7)
cv.carseats <- cv.tree(tree.carseats, FUN = prune.misclass)
```

► Code

```
$size
[1] 21 19 14  9  8  5  3  2  1

$dev
[1] 75 75 75 74 82 83 83 85 82

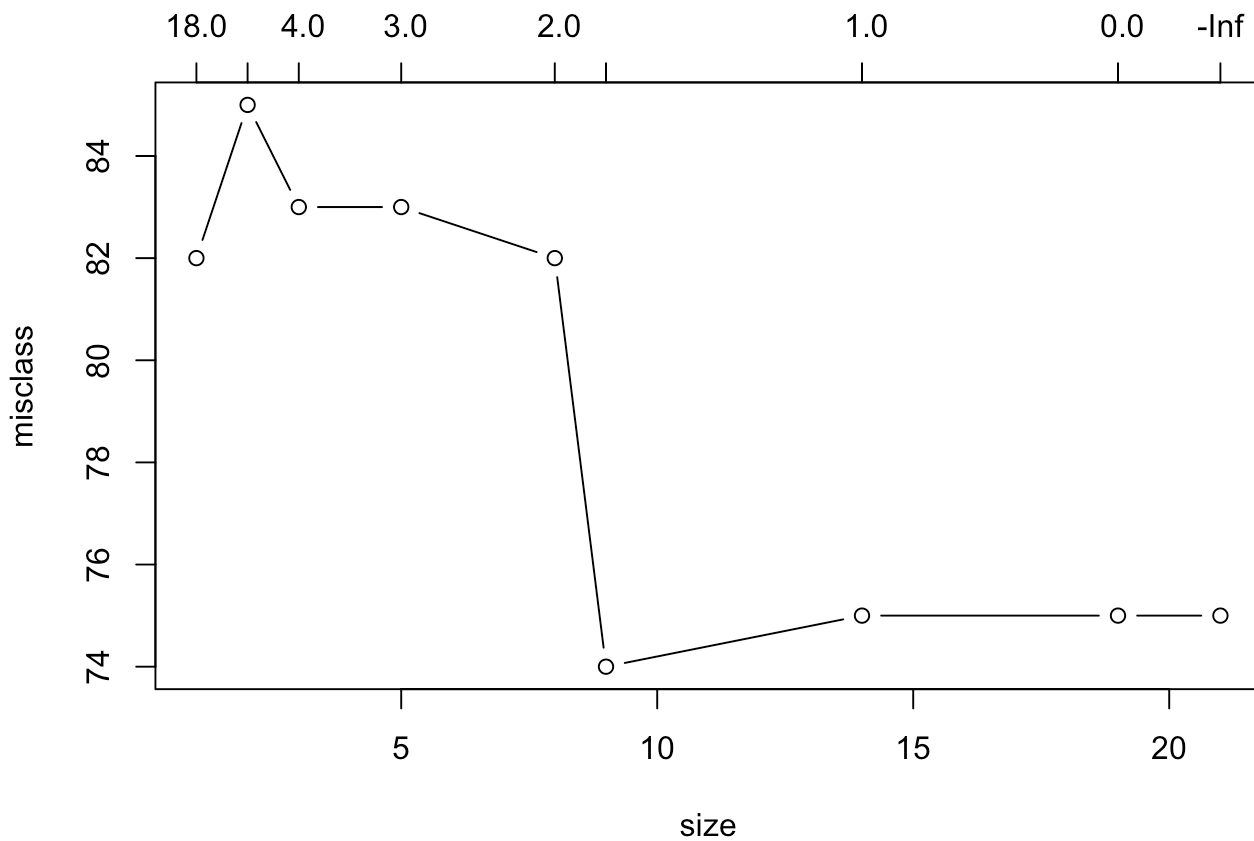
$k
[1] -Inf  0.0  1.0  1.4  2.0  3.0  4.0  9.0 18.0

$method
[1] "misclass"

attr("class")
[1] "prune"      "tree.sequence"
```

Despite its name, dev corresponds to the number of cross-validation errors. The tree with 9 terminal nodes results in only 74 cross-validation errors. We plot the error rate as a function of both $size$ and k .

```
plot(cv.carseats, type="b")
```



We now apply the `prune.misclass()` function in order to prune the tree to obtain the 9-node tree.

```
prune.carseats <- prune.misclass(tree.carseats, best = 9)
plot(prune.carseats, type = "uniform")
text(prune.carseats, pretty = 0)
```

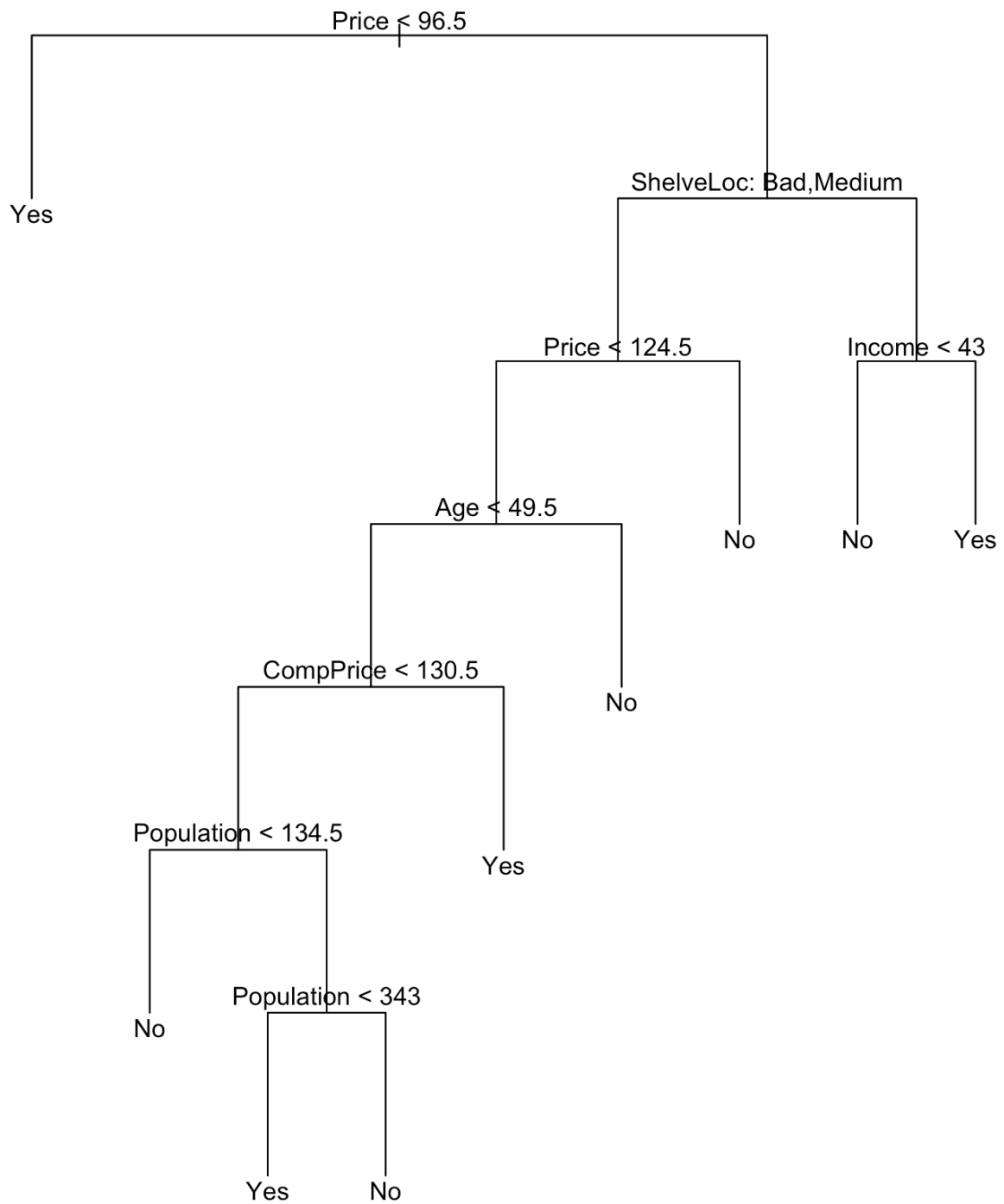


Figure 5

Prediction Performance

Let's see how well this prune tree performed on the test set. The `predict()` function can with the argument `type = "class"` to instruct R to return the actual class prediction (otherwise a matrix of probabilities will be returned)

```
tree.pred <- predict(prune.carseats, Carseats.test,
  type = "class")
(pruned.tab <- table(tree.pred, High.test))
```

```
      High.test
tree.pred No Yes
No      97  25
Yes     20  58
```

We see that 97 + 58 of the test observations are correctly classified (i.e. accuracy of 77.5% test accuracy)

For fun, let's compare this to the performance of the unpruned tree.

```
tree.pred <- predict(tree.carseats, Carseats.test,
  type = "class")
(unpruned.tab <- table(tree.pred, High.test))
```

```
      High.test
tree.pred No Yes
No     104  31
Yes     13  52
```

```
pctest.acc = (unpruned.tab[1,1] + unpruned.tab[2,2])/sum(unpruned.tab)
```

We see that 104 + 52 of the test observations are correctly classified (i.e. accuracy of 78% test accuracy). So not only has the pruning process produced a more interpretable tree, but it has also slightly improved the classification accuracy.

Example: Chemical Manufacturing Process Data

See `?ChemicalManufacturingProcess` for details on the dataset.

Response: `Yield` (continuous -> regression tree)

```
library(AppliedPredictiveModeling)
data(ChemicalManufacturingProcess)
ChemicalManufacturingProcess
```

	Yield <dbl>	BiologicalMaterial01 <dbl>	BiologicalMaterial02 <dbl>	BiologicalMaterial03 <dbl>
1	38.00	6.25	49.58	56.97
2	42.44	8.01	60.97	67.48
3	42.03	8.01	60.97	67.48
4	41.42	8.01	60.97	67.48

	Yield	BiologicalMaterial01	BiologicalMaterial02	BiologicalMaterial03
	<dbl>	<dbl>	<dbl>	<dbl>
5	42.49	7.47	63.33	72.25
6	43.57	6.12	58.36	65.31
7	43.12	7.48	64.47	72.41
8	43.06	6.94	63.60	72.06
9	41.49	6.94	63.60	72.06
10	42.45	6.94	63.60	72.06

1-10 of 176 rows | 1-5 of 59 columns

Previous **1** [2](#) [3](#) [4](#) [5](#) [6](#) ... [18](#) [Next](#)

Train/Test

```
set.seed(1)

n <- nrow(ChemicalManufacturingProcess)
train_idx <- sample(1:n, size = floor(0.7 * n))
test_idx <- setdiff(1:n, train_idx)

train_dat <- ChemicalManufacturingProcess[train_idx, ]
test_dat <- ChemicalManufacturingProcess[test_idx, ]
```

Fit a regression tree

```
tree.cmp <- tree(
  Yield ~ .,
  data = train_dat
)

summary(tree.cmp)
```

Regression tree:

```
tree(formula = Yield ~ ., data = train_dat)
```

Variables actually used in tree construction:

```
[1] "ManufacturingProcess13" "BiologicalMaterial08" "ManufacturingProcess20"
[4] "ManufacturingProcess32" "ManufacturingProcess09" "BiologicalMaterial03"
[7] "BiologicalMaterial05" "ManufacturingProcess21" "ManufacturingProcess33"
[10] "ManufacturingProcess31" "ManufacturingProcess42" "ManufacturingProcess23"
```

Number of terminal nodes: 13

Residual mean deviance: 0.6321 = 61.31 / 97

Distribution of residuals:

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-2.24400	-0.47740	-0.06835	0.00000	0.43100	2.02800

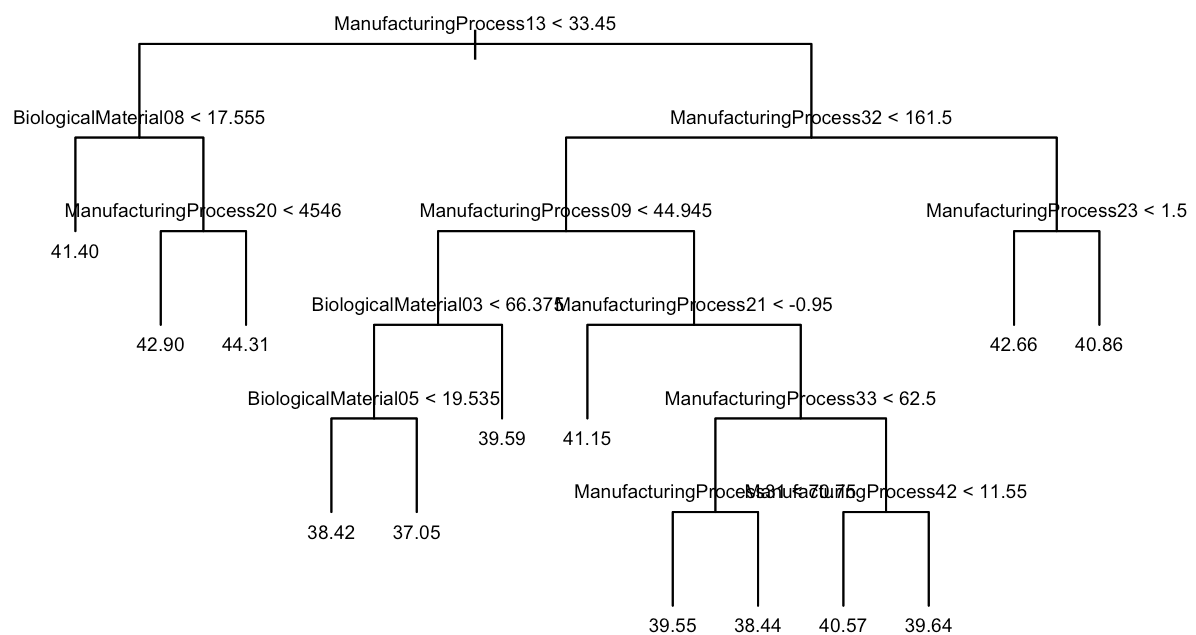
Plot


```

plot(
  tree.cmp,
  type = "uniform"
)

text(
  tree.cmp,
  pretty = 0,
  cex = 0.6
)

```



CV for choosing size of tree

```

set.seed(2)

cv.cmp <- cv.tree(
  tree.cmp,
  FUN = prune.tree
)

cv.cmp

```

```
$size
```

```
[1] 13 12 11 10 9 8 7 6 5 4 3 2 1
```

```
$dev
```

```
[1] 270.8356 283.6175 295.5806 295.0836 290.7608 267.5716 279.3550 282.1140  
[9] 294.1859 296.3885 287.4227 318.9326 407.1725
```

```
$k
```

```
[1]      -Inf    4.264531    6.112893    6.497273    7.641846   11.018505  
[7]  12.141002  13.076930  15.669458  18.941400  30.602821  66.166707  
[13] 151.298045
```

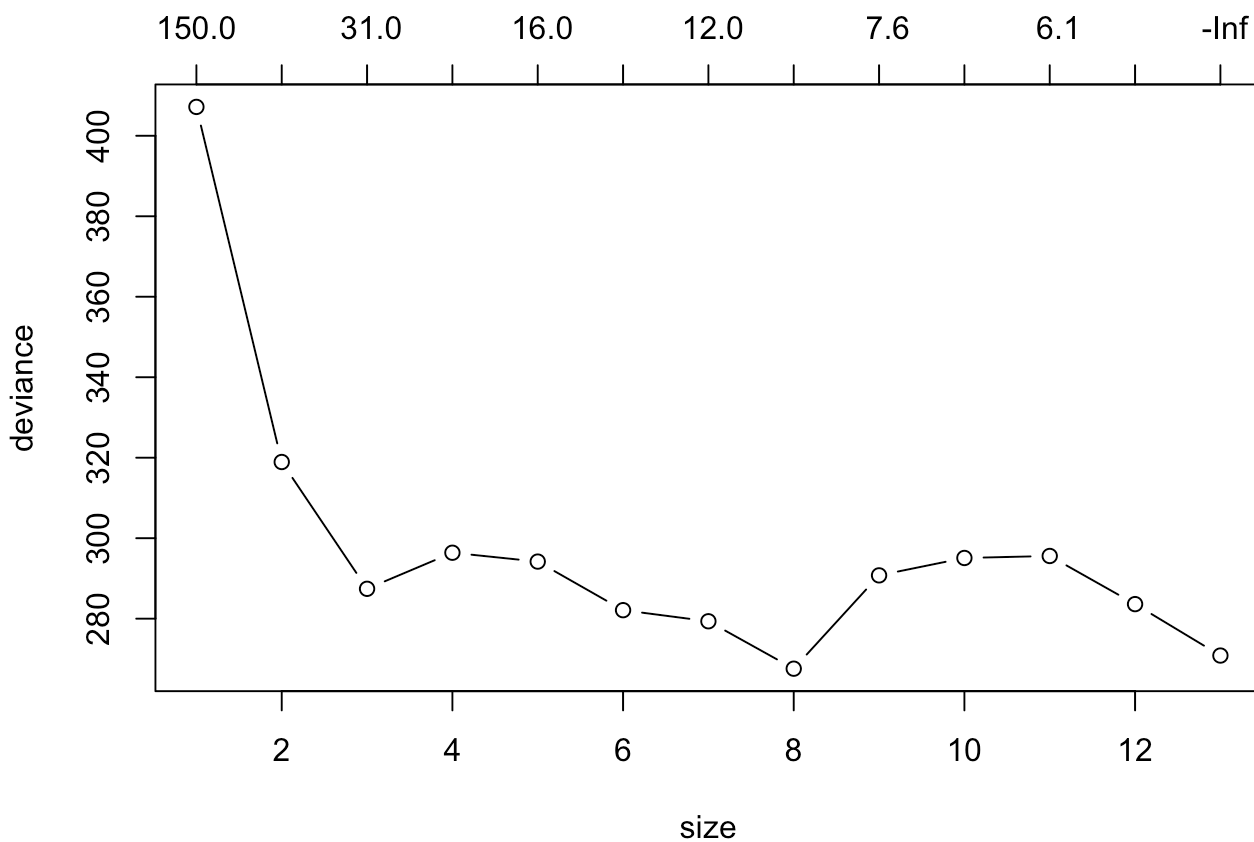
```
$method
```

```
[1] "deviance"
```

```
attr("class")
```

```
[1] "prune"          "tree.sequence"
```

```
plot(cv.cmp, type= "b")
```



Choose optimal size:

```
best_size <- cv.cmp$size[which.min(cv.cmp$dev)]
```

```
best_size
```

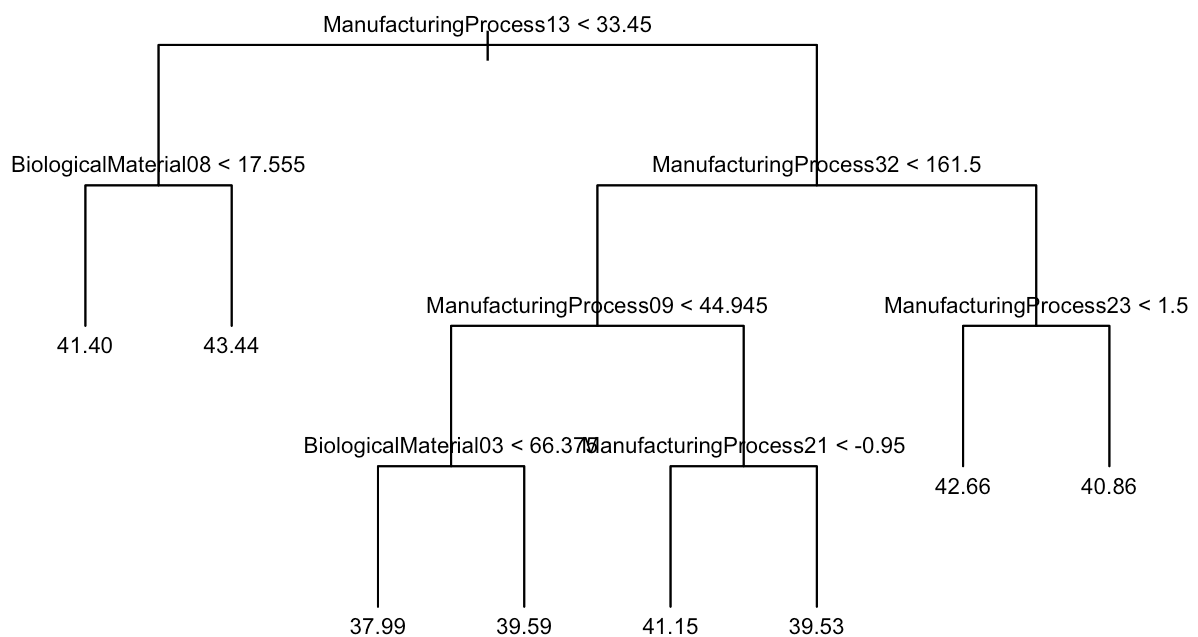
```
[1] 8
```

Prune the tree

```
pruned.cmp <- prune.tree(
  tree.cmp,
  best = best_size
)
```

Plot the pruned tree

```
plot(pruned.cmp, type="uniform")
text(pruned.cmp, pretty = 0, cex = 0.7)
```



Test error after pruning

```
# pruned tree metrics
pruned_pred <- predict(pruned.cmp, newdata = test_dat)
test_mse_pruned <- mean((test_dat$Yield - pruned_pred)^2)

# compare with unpruned tree
```

```
tree_pred <- predict(tree.cmp, newdata = test_dat)
test_mse_full <- mean((test_dat$Yield - tree_pred)^2)

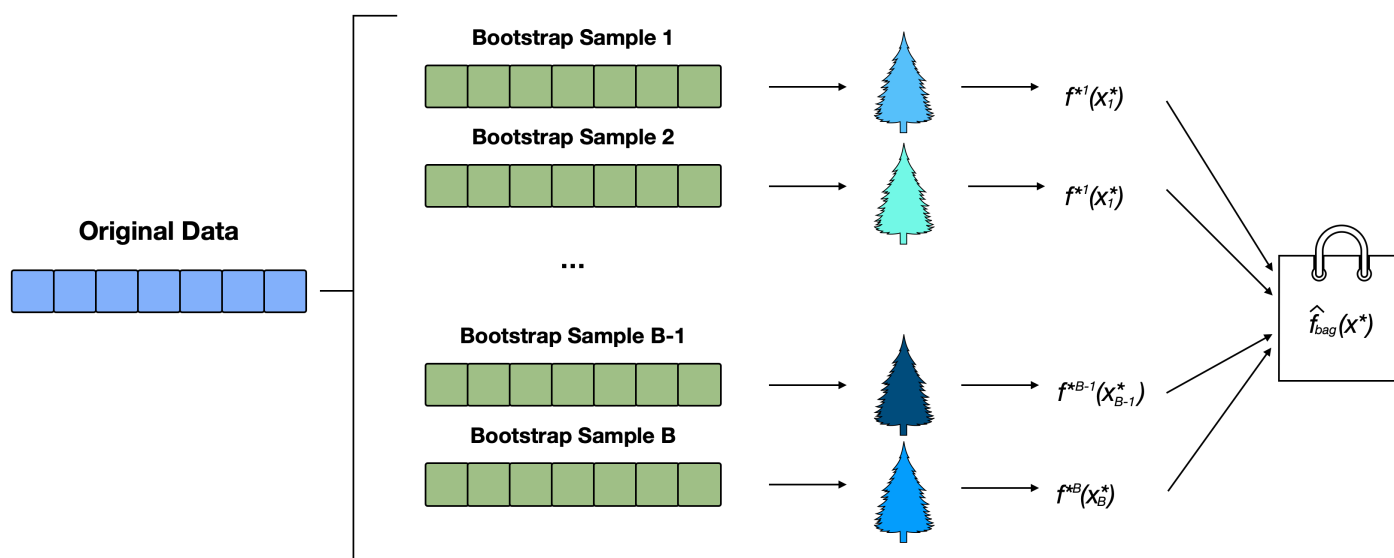
c(
  Full_tree    = test_mse_full,
  Pruned_tree  = test_mse_pruned
)
```

```
Full_tree Pruned_tree
2.491140  2.359624
```

Ensemble Methods

The limitations of CART motivate **ensemble methods**, which combine many trees to improve performance.

Bagging and Random Forests



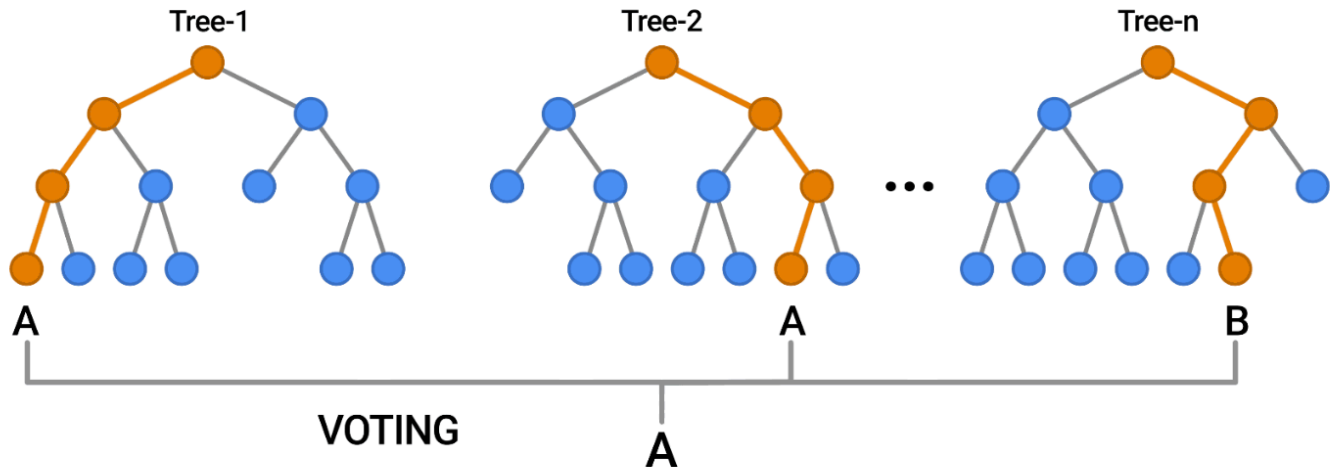
Bagging (short for *bootstrap aggregation*) can be used to improve simple CART by reducing its variance.

Steps for Bagging

1. Create B bootstrap samples
2. Fit a (bushy) tree to each bootstrap sample
3. Each decision tree is used to make a prediction.
4. Aggregate the B predictions:
 - for regression predictions are typically averaged
 - for classification we use a majority vote.

Random Forest

EXAMPLES



Random Forests uses a similar method to bagging, however, we only consider m predictors at each split

- default for classification is $m = \sqrt{p}$
- default for regression is $m = p/3$

Objective: Extend bagging by introducing additional randomness through random feature selection at each split during the construction of decision trees.

Benefits:

- Further reduction of overfitting compared to bagging.
- Improved generalization and accuracy.

Steps for RF

1. Create B bootstrap samples
2. Fit a tree to each bootstrap sample, HOWEVER, when creating nodes in each decision tree, only a random subset of m features is considered for splitting at each node.
3. Each decision tree is used to make a prediction. Aggregate the B predictions.

This modification reduces the variance even more than bagging which tend to improve performance.

Both are fit with the same function in R:

```
randomForest(formula, data=NULL, mtry, importance = FALSE)
```

- when `mtry` = p we get bagging and when `mtry` < p we get random forests. Note that the default values are different for classification ($\sqrt{p}$) where p is number of predictors) and regression ($p/3$)
- when `importance` = `TRUE` we get variable importance computations (don't worry about *how* they are calculated)

Model assessment

- The out-of-bag (OOB) samples (approximately 1/3 of the data) are used to create an cross-validation like estimate of error
- These are outputted by the `randomForest()` function

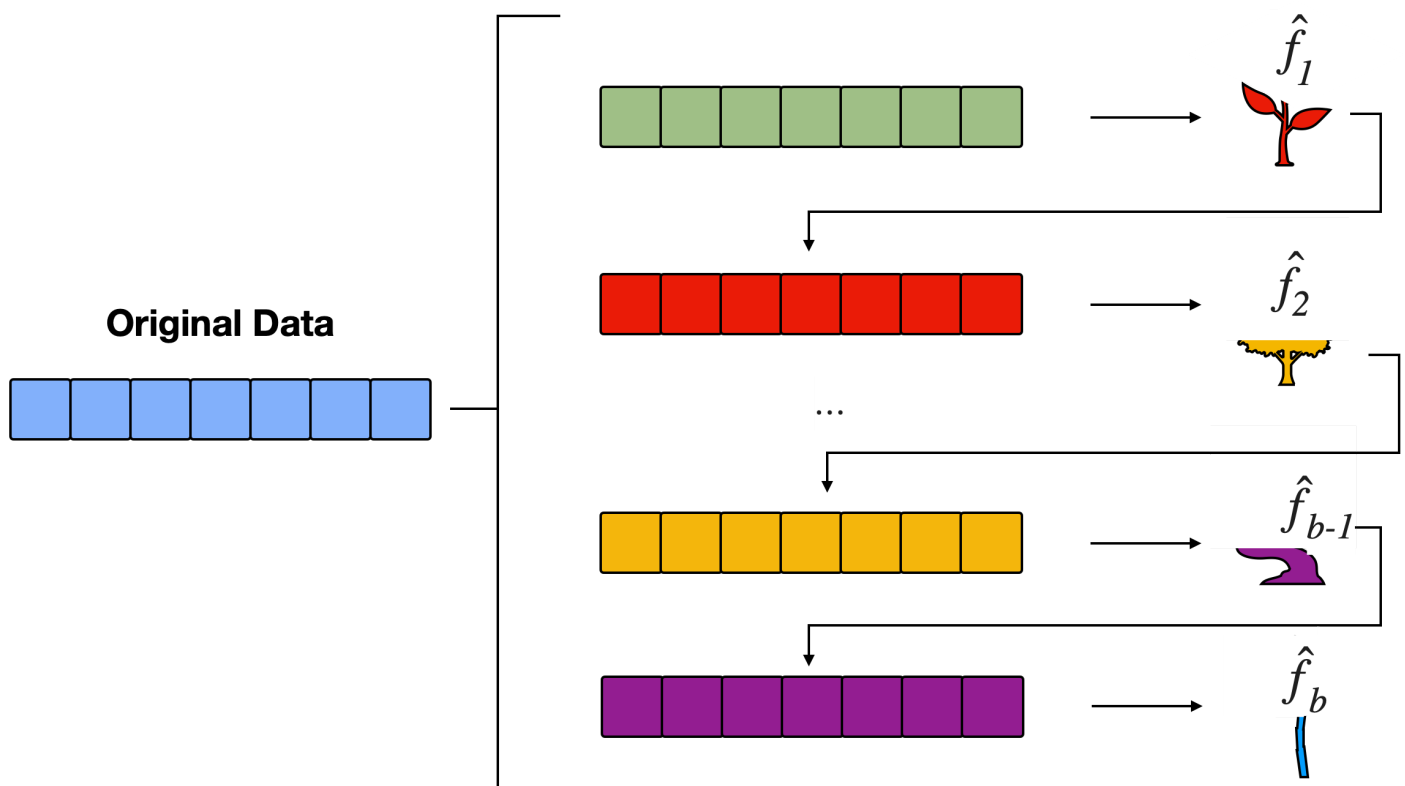
Pro:

- this approach prevents the risk of overfitting
- we get OOB error estimates “for free”
- makes trees a competitive ML algorithm
- Increased robustness and stability over single Classification decision tree.
- Improved generalization by reducing variance.
- Feature importance analysis.

Con:

- we’ve lost all interpretability
- takes longer to run

Boosting



Builds a strong model sequentially by focusing on correcting errors in the ensemble.

Definition: Boosting is an ensemble learning method that combines the strengths of multiple weak learners to create a strong predictive model.

Objective: Build a strong predictive model by sequentially training weak learners (typically shallow trees), with each new learner focusing on correcting the errors of the ensemble so far.

Key Concepts

- **Weak Learner:** A model that performs slightly better than random guessing. Decision stumps (one-level decision trees) are common weak learners.
- **Strong Learner:** A composite model that achieves high accuracy by aggregating multiple weak learners.
- **Sequential Learning:** Models are trained one after another, with each new model emphasizing the mistakes of the previous ones.

Steps for Boosting

1. **Initialization:** Start with a base model (e.g., a simple decision tree) and assign equal weights to all training samples.
2. **Iterative Training:**
 - **Training:** Train a weak learner on the weighted data.
 - **Evaluation:** Assess the learner's performance and calculate its error rate.
 - **Weight Update:** Increase the weights of misclassified instances to focus the next learner on harder cases.
 - **Aggregation:** Combine the learners' predictions, typically using a weighted majority vote (classification) or weighted sum (regression).
3. **Final Model:** The ensemble of all weak learners, weighted by their accuracy, forms the final strong model.

Advantages of Boosting

- **Improved Accuracy:** Combines multiple models to achieve higher predictive performance.
- **Flexibility:** Can be applied to various types of models and loss functions.
- **Handling Complex Patterns:** Effective in capturing intricate relationships in data.
- **Reduction of Bias and Variance:** Balances the trade-off between bias (underfitting) and variance (overfitting).

Disadvantages of Boosting

- **Sensitivity to Noisy Data:** Boosting can overemphasize outliers and noisy instances, leading to overfitting.
- **Computationally Intensive:** Especially with large datasets and numerous iterations.
- **Requires Careful Tuning:** Hyperparameters such as the number of learners, learning rate, and tree depth need to be optimized
- No more OOB estimates or variable importance

Boosting vs. Bagging

Boosting:

- **Approach:** Sequential training; focuses on correcting errors of previous models.
- **Goal:** Reduce both bias and variance.

Bagging (Bootstrap Aggregating):

- **Approach:** Parallel training on different subsets of data; aggregates predictions.
- **Goal:** Primarily reduce variance; less effective in reducing bias.
- **Key Difference:** Boosting emphasizes sequential improvement, while bagging emphasizes parallel diversity.

Hyperparameter Tuning in Boosting

- **Number of Estimators:** Determines the number of weak learners to combine.
- **Learning Rate:** Controls the contribution of each learner; lower rates require more learners.
- **Tree Depth:** Limits the complexity of each weak learner; deeper trees can capture more complex patterns but may overfit.
- **Regularization Parameters:** Techniques like shrinkage, subsampling, and column sampling to prevent overfitting.

R functions:

```
gbm(formula, distribution="bernoulli", n.trees = B, interaction.depth = d , shrinkage = l
```

from the **gmb** package. General expectation of performance:

Single Tree < Bagging < RF < Boosting

Example

```
library(gbm)
```

Loaded gbm 2.2.2

This version of gbm is no longer under development. Consider transitioning to gbm3, <https://github.com/gbm-developers/gbm3>

```
library(randomForest)
```

randomForest 4.7-1.2

Type `rfNews()` to see new features/changes/bug fixes.

```
set.seed(1)

n <- nrow(ChemicalManufacturingProcess)
train_idx <- sample(1:n, size = floor(0.7 * n))
test_idx <- setdiff(1:n, train_idx)

train_dat <- ChemicalManufacturingProcess[train_idx, ]
test_dat <- ChemicalManufacturingProcess[test_idx, ]

# remove rows with missing values
train_dat <- na.omit(train_dat)
test_dat <- na.omit(test_dat)
```

Bagging

```
set.seed(1)

p <- ncol(train_dat) - 1 # number of predictors

bag.cmp <- randomForest(
  Yield ~ .,
  data = train_dat,
  mtry = p,
  ntree = 500,
  importance = TRUE
)

bag.cmp
```

Call:

```
randomForest(formula = Yield ~ ., data = train_dat, mtry = p, ntree = 500,
importance = TRUE)
```

Type of random forest: regression

Number of trees: 500

No. of variables tried at each split: 57

Mean of squared residuals: 1.249183

% Var explained: 66.05

Test MSE

```
bag_pred <- predict(bag.cmp, newdata = test_dat)
mse_bag <- mean((test_dat$Yield - bag_pred)^2)
mse_bag
```

[1] 1.330471

varImpPlot(bag.cmp)

bag.cmp

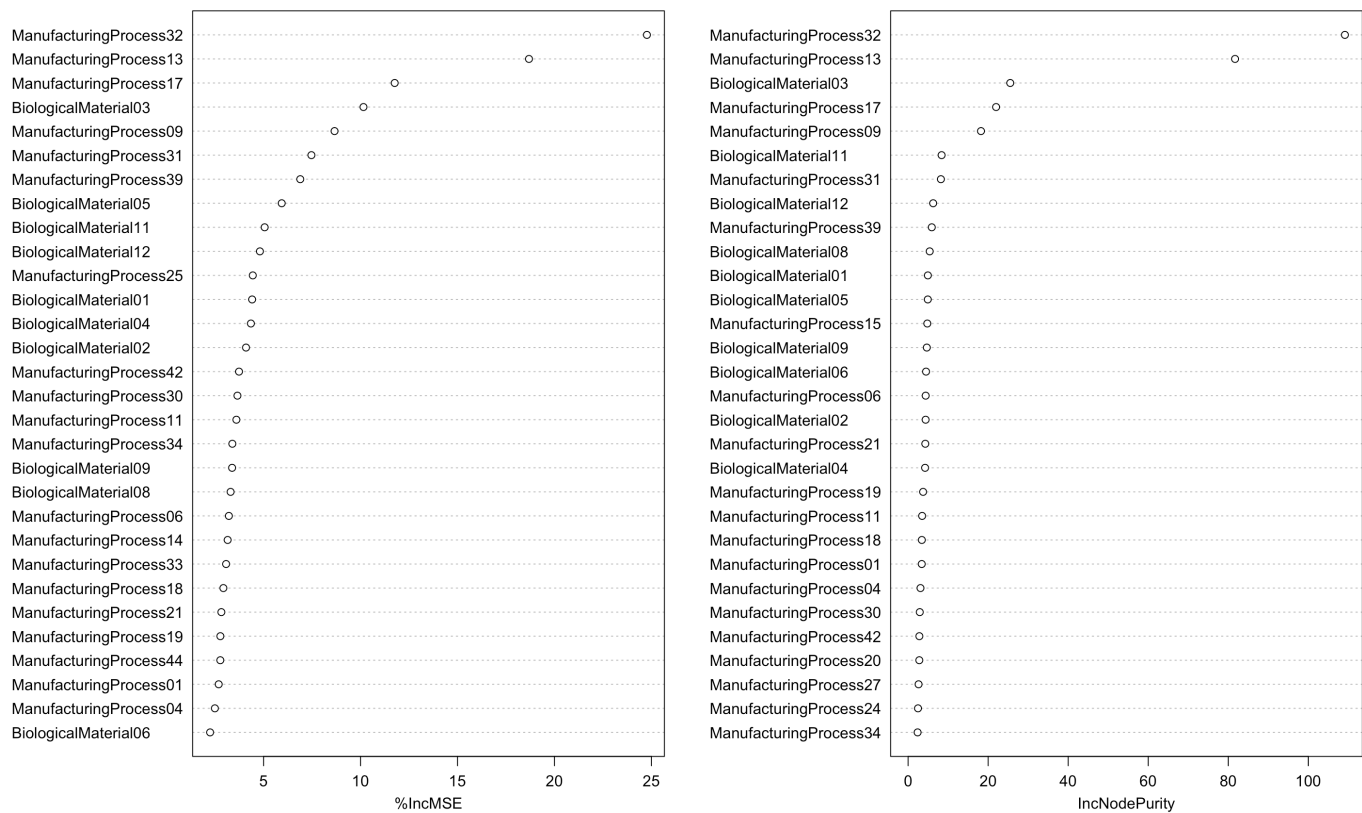


Figure 6

Random Forests

```
set.seed(1)

rf.cmp <- randomForest(
  Yield ~ .,
  data = train_dat,
  mtry = floor(sqrt(p)), # standard default for regression
  ntree = 500,
  importance = TRUE
)

rf.cmp
```

Call:

```
randomForest(formula = Yield ~ ., data = train_dat, mtry = floor(sqrt(p)), ntree =
500, importance = TRUE)
```

Type of random forest: regression

Number of trees: 500

No. of variables tried at each split: 7

Mean of squared residuals: 1.42042

% Var explained: 61.4

Test MSE

```
rf_pred <- predict(rf.cmp, newdata = test_dat)

mse_rf <- mean((test_dat$Yield - rf_pred)^2)
mse_rf
```

[1] 1.26618

```
varImpPlot(rf.cmp)
```

rf.cmp

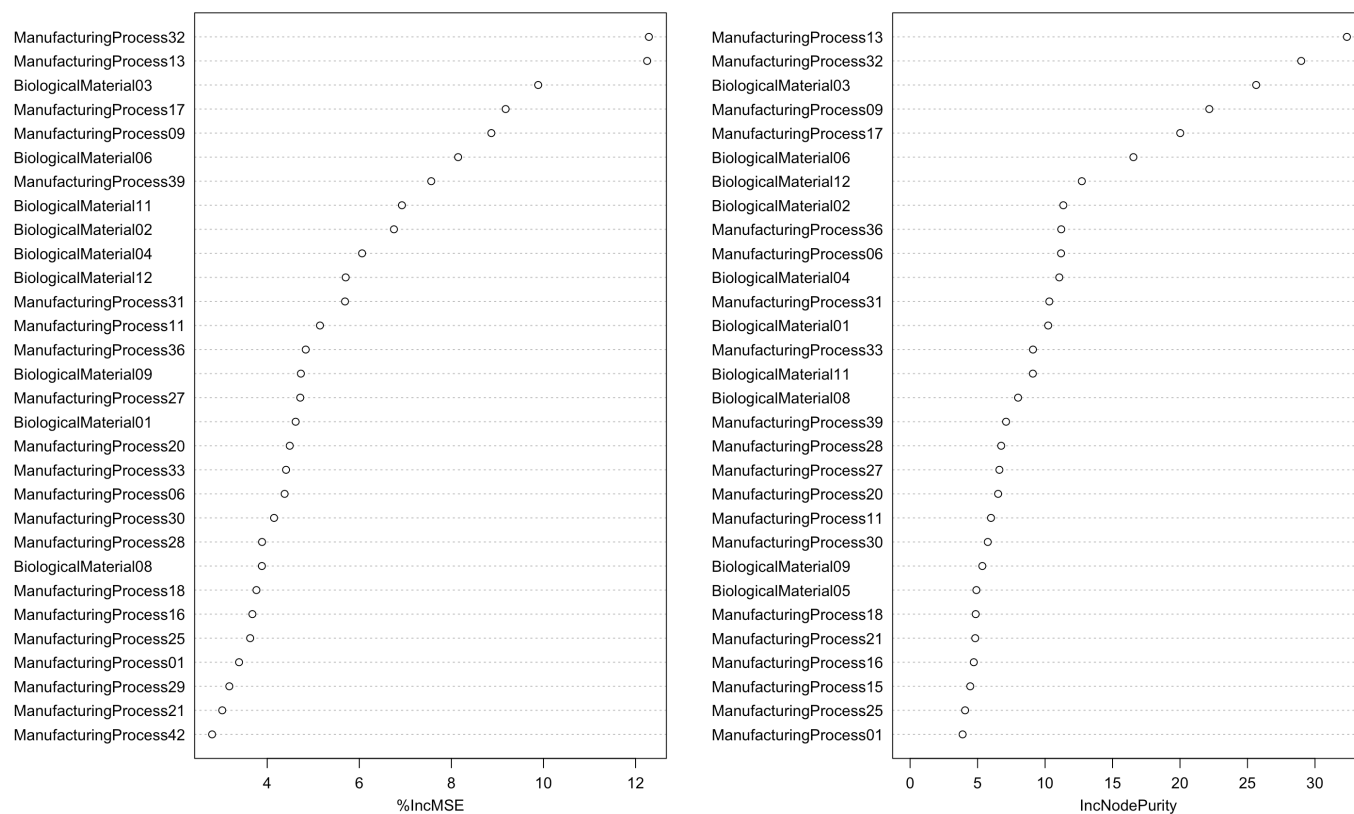


Figure 7

Boosting

```
set.seed(1)
```

```
boost.cmp <- gbm(
  Yield ~ .,
  data = train_dat,
  distribution = "gaussian",
  n.trees = 3000,
  interaction.depth = 2,    # shallow trees
  shrinkage = 0.01,
  bag.fraction = 0.7,
  verbose = FALSE
)

summary(boost.cmp)
```

	var <chr>	rel.inf <dbl>
ManufacturingProcess32	ManufacturingProcess32	23.164395228
ManufacturingProcess13	ManufacturingProcess13	17.730579298
ManufacturingProcess09	ManufacturingProcess09	7.100143156
BiologicalMaterial03	BiologicalMaterial03	6.928917825
ManufacturingProcess06	ManufacturingProcess06	3.736059067
ManufacturingProcess31	ManufacturingProcess31	3.549669430
ManufacturingProcess17	ManufacturingProcess17	3.211909837
ManufacturingProcess39	ManufacturingProcess39	3.093572447
BiologicalMaterial09	BiologicalMaterial09	2.902187346
BiologicalMaterial05	BiologicalMaterial05	1.931642506

1-10 of 57 rows

Previous **1** [2](#) [3](#) [4](#) [5](#) [6](#) [Next](#)

Figure 8

```
boost.cmp
```

```
gbm(formula = Yield ~ ., distribution = "gaussian", data = train_dat,
     n.trees = 3000, interaction.depth = 2, shrinkage = 0.01,
     bag.fraction = 0.7, verbose = FALSE)
```

A gradient boosted model with gaussian loss function.

3000 iterations were performed.

There were 57 predictors of which 55 had non-zero influence.

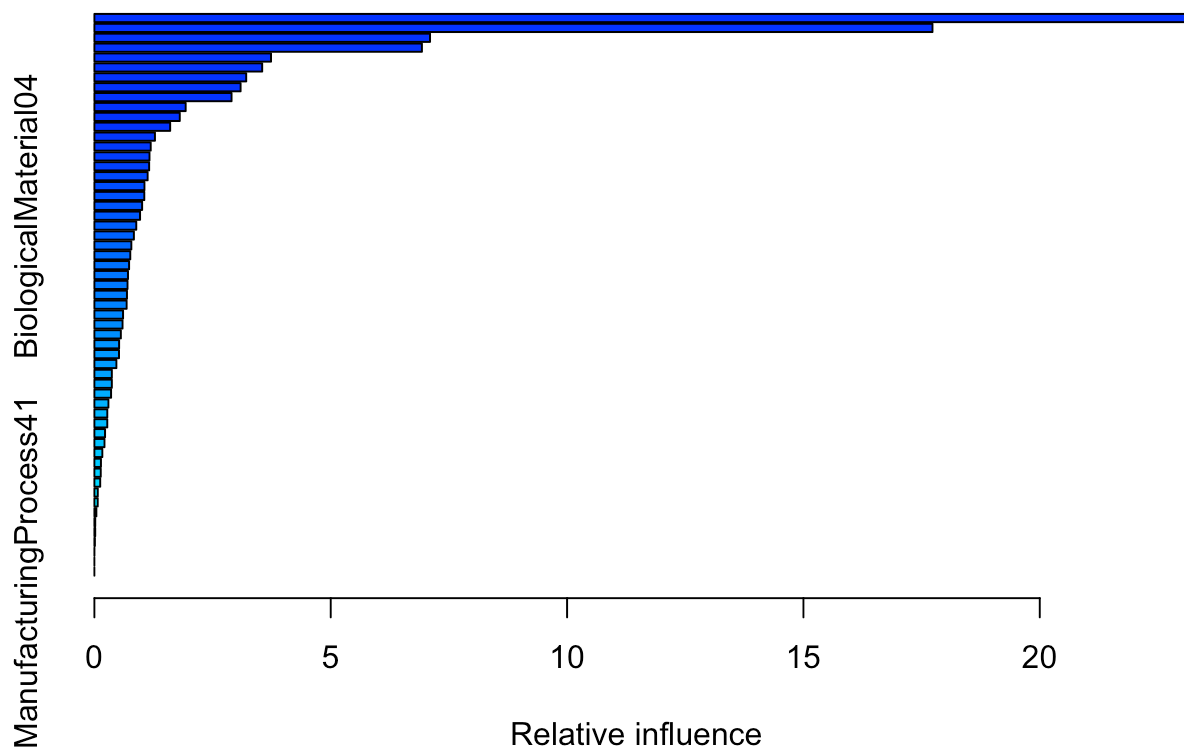


Figure 9

Test MSE

```
boost_pred <- predict(
  boost.cmp,
  newdata = test_dat
)
```

Using 3000 trees...

```
mse_boost <- mean((test_dat$Yield - boost_pred)^2)
mse_boost
```

[1] 1.164141

Test set MSE comparison for CART and tree-based ensemble methods.

Method	Test_MSE
Pruned CART	2.360
Bagging	1.330
Random Forest	1.266

Method	Test_MSE
Boosting	1.164

Regularization techniques

Ridge Regression and LASSO (Least Absolute Shrinkage and Selection Operator) are regularization techniques used in linear regression to prevent overfitting and handle multicollinearity. Both methods add a regularization term to the linear regression objective function, influencing the model coefficients during training.

Ridge Regression (L2 Regularization):

- **Model:** A regularized version of linear regression that adds a penalty term proportional to the square of the magnitude of the coefficients.
- **Objective:** Minimizes the sum of squared differences with the addition of a regularization term.

$$\text{Minimize } \mathcal{L}(\beta) = \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

- **Regularization Term:** $\lambda \sum_{j=1}^p \beta_j^2$
- **Purpose:**
 - Prevents overfitting by shrinking large coefficients, particularly useful when multicollinearity is present.
 - Does not lead to exact zero coefficients; instead, it shrinks all coefficients proportionally.
- **Effect on Coefficients:**
 - Shrinks coefficients closer to zero but does not set any coefficients exactly to zero.
 - Does not perform feature selection directly.
- **Hyperparameter:** λ controls the strength of regularization.
 - A larger λ results in more shrinkage of the coefficients.
 - A smaller λ results in a model closer to standard linear regression.
- **Use Cases:** Helps prevent overfitting, especially when the number of predictors is large or when there is multicollinearity among the features.

Lasso Regression (L1 Regularization):

- **Model:** Similar to Ridge regression but adds a penalty term proportional to the absolute values of the coefficients.
- **Objective:** Minimizes the sum of squared differences with the addition of a regularization term.

$$\text{Minimize } J(\beta) = \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- **Regularization Term:** $\lambda \sum_{j=1}^p |\beta_j|$
- **Purpose:**
 - Promotes sparsity by encouraging exact zero coefficients.
 - Effective for feature selection, automatically setting some coefficients to zero.
- **Effect on Coefficients:**
 - Can lead to exact zero coefficients, effectively removing irrelevant features.
- **Hyperparameter:** λ controls the strength of regularization.
- **Use Cases:** Useful for feature selection as it tends to drive some coefficients to exactly zero.

Key difference: Ridge Regression shrinks coefficients towards zero whereas Lasso can lead to exact zero coefficients, performing feature selection.

You should know

- Know how to read the output from the glmnet package
- How to interpret the resulting models
- how to select the amount of regularization

Example:

Train/Test was done earlier but we do need this package now:

```
library(glmnet)
```

Loading required package: Matrix

Loaded glmnet 4.1-8

Wrangling

Ridge/LASSO (via glmnet) can't handle **NAs**. In **ChemicalManufacturingProcess**, missing values are spread across many predictors. Even if each variable has only a few missing entries, removing rows with any **NA** can discard a large fraction of the dataset. So imputation is usually the better choice here. Here is an example of median imputation within the training set and apply the same medians to the test set.

```
# simple numeric median imputation
impute_median <- function(train_df, test_df) {
  for (j in seq_len(ncol(train_df))) {
```

```
  if (is.numeric(train_df[[j]])) {
    med <- median(train_df[[j]], na.rm = TRUE)
    train_df[[j]][is.na(train_df[[j]])] <- med
    test_df[[j]][is.na(test_df[[j]])] <- med
  }
}
list(train = train_df, test = test_df)
}

imp <- impute_median(train_dat, test_dat)
train_imp <- imp$train
test_imp <- imp$test
```

We use `model.matrix()` so glmnet gets an X matrix without the intercept column.

```
# response
y_train <- train_imp$Yield
y_test <- test_imp$Yield

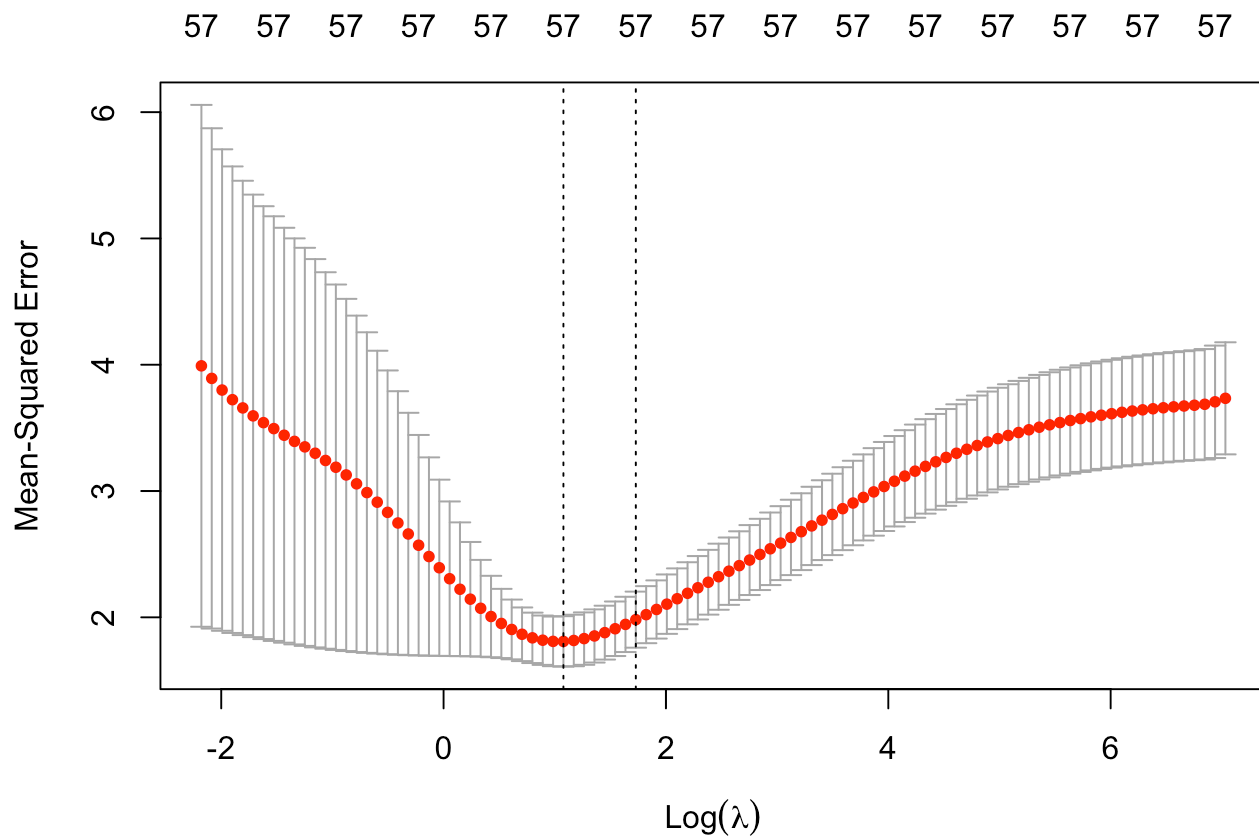
# predictors (drop intercept)
X_train <- model.matrix(Yield ~ ., data = train_imp)[, -1]
X_test <- model.matrix(Yield ~ ., data = test_imp)[, -1]
```

Ridge

```
set.seed(1)

ridge_cv <- cv.glmnet(
  X_train, y_train,
  alpha = 0,          # ridge
  standardize = TRUE
)

plot(ridge_cv)
```

```
lambda_ridge_min <- ridge_cv$lambda.min
lambda_ridge_1se <- ridge_cv$lambda.1se
```

```
lambda_ridge_min
```

```
[1] 2.938407
```

```
lambda_ridge_1se
```

```
[1] 5.635602
```

Test MSE using λ_{min}

```
ridge_pred <- predict(ridge_cv, s = lambda_ridge_min, newx = X_test)
mse_ridge <- mean((y_test - ridge_pred)^2)
mse_ridge
```

```
[1] 1.257673
```

Test MSE using λ_{1se}

```
# Ridge predictions using lambda.1se
ridge_pred_1se <- predict(
  ridge_cv,
  s = ridge_cv$lambda.1se,
  newx = X_test
)

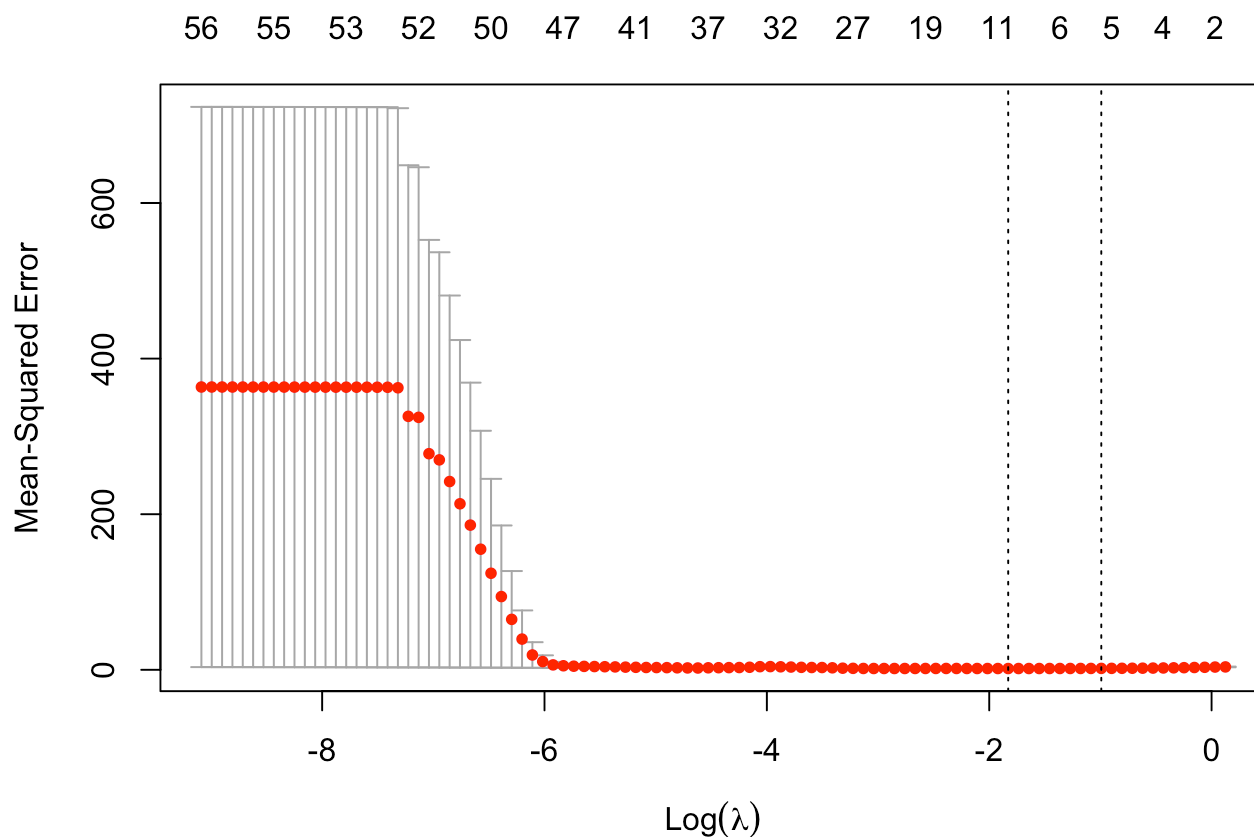
mse_ridge_1se <- mean((y_test - ridge_pred_1se)^2)
mse_ridge_1se
```

[1] 1.46807

LASSO

```
set.seed(1)

lasso_cv <- cv.glmnet(
  X_train, y_train,
  alpha = 1,          # LASSO
  standardize = TRUE
)
plot(lasso_cv)
```



```
lambda_lasso_min <- lasso_cv$lambda.min
lambda_lasso_1se <- lasso_cv$lambda.1se

lambda_lasso_min
```

```
[1] 0.160504
```

```
lambda_lasso_1se
```

```
[1] 0.370785
```

Test MSE using λ_{min}

```
lasso_pred <- predict(lasso_cv, s = lambda_lasso_min, newx = X_test)
mse_lasso <- mean((y_test - lasso_pred)^2)
mse_lasso
```

```
[1] 1.222653
```

Test MSE using λ_{1se}

```
# LASSO predictions using lambda.1se
lasso_pred_1se <- predict(
  lasso_cv,
  s = lasso_cv$lambda.1se,
  newx = X_test
)

mse_lasso_1se <- mean((y_test - lasso_pred_1se)^2)
mse_lasso_1se
```

```
[1] 1.418973
```

Compare all four penalized models

```
perf_penalized <- data.frame(
  Method = c(
    "Ridge (lambda.min)",
    "Ridge (lambda.1se)",
    "LASSO (lambda.min)",
    "LASSO (lambda.1se)"
  ),
  Test_MSE = c(
    mse_ridge,
    mse_ridge_1se,
    mse_lasso,
    mse_lasso_1se
  )
)
```

```
)
```

```
perf_penalized
```

Method	Test_MSE
<chr>	<dbl>
Ridge (lambda.min)	1.257673
Ridge (lambda.1se)	1.468070
LASSO (lambda.min)	1.222653
LASSO (lambda.1se)	1.418973

4 rows

Footnotes

1. The values are stored in `best_k_vec` : 6, 13, 13, 17, 19, 20, 28, 32, 33, 40 ↩